

Volumetric Data Structures for Ray Marching Clouds on the GPU

Implementation and Comparative Analysis of a Dense Grid, a
Two-Level Sparse Grid, and NanoVDB

Simon Nützl

Thesis for the attainment of the academic degree

Bachelor of Science (B.Sc.)

at the TUM School of Computation, Information and Technology of the Technical University of Munich.

Examiner:

Prof. Dr. Rüdiger Westermann

Supervisor:

Dr. rer. nat. Ludwic Leonard Mendez

Submitted:

Munich, 14.09.2026

I hereby declare that this thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references. I additionally used AI tools to assist with grammar and language polishing throughout the text, and to help understand and document the usage of the framework's methods, which lacked official documentation. I have reviewed and take full responsibility for all content in this thesis.

Abstract

Volumetric clouds are costly to render because ray marching must repeatedly sample a three-dimensional density field, and the way this field is stored in memory strongly influences GPU performance. Dense grids are simple to query but waste memory and bandwidth on empty space, while hierarchical sparse structures such as OpenVDB avoid this waste at the cost of pointer indirection and traversal overhead. This thesis investigates whether a custom two-level sparse grid can combine the low, branch-free lookup cost of a dense grid with the empty-space compression of a hierarchical structure, and whether NanoVDB’s architectural generality, designed for large, unbounded scenes, still pays off when applied to a single static cloud rendered from a fixed camera.

Four volumetric representations are implemented as GLSL shaders within a shared rendering pipeline: a dense grid, a two-level grid, a padded variant of the two-level grid, and NanoVDB. Each is evaluated on three clouds of increasing occupancy (21.5% to 37.5%) in terms of memory footprint, point-query throughput under coherent and incoherent access, and rendering performance under both fixed-step and space-skipping ray marching.

The results show that no single representation dominates every metric. The unpadded two-level grid is the most compact representation, requiring roughly half the memory of the dense grid, while the padded variant is the fastest representation across every timing benchmark, outperforming both the dense grid and NanoVDB. NanoVDB, in contrast, is the slowest representation in every timing discipline, and its memory footprint falls between the two two-level variants rather than surpassing them. These findings indicate that, for static, bounded cloud volumes, a simple two-level sparse grid matches or exceeds NanoVDB’s compression while avoiding its performance penalty, and that OpenVDB’s architectural advantages provide no measurable benefit when the generality they enable is not required.

Contents

Abstract	iii
1 Introduction	1
1.1 Problem Statement and Motivation	1
1.2 Research Question and Thesis Scope	1
1.3 Thesis Outline	2
2 Background	3
2.1 Volumetric Data	3
2.1.1 Dense Representations	4
2.1.2 Bricked Representations	5
2.1.3 OpenVDB	6
2.2 Traversals	8
2.2.1 Ray Bounding Box Intersection	8
2.2.2 Absorption-Only Ray Marching	9
2.2.3 Space-Skipping via Digital Differential Analyzers	10
2.2.4 Trilinear Interpolation	11
2.3 GPU-Specific Bottlenecks	12
3 Implementation	14
3.1 Architectural Overview	14
3.2 Point Sampler Implementations	15
3.2.1 Dense Grid Sampler	15
3.2.2 Two-Level Grid Sampler	16
3.2.3 NanoVDB Sampler	18
3.3 Traversal & Sampling Strategies	20
3.3.1 Ray-AABB Intersection	20
3.3.2 Fixed-Step Ray Marching	21
3.3.3 Space-Skipping Ray Marching with DDA	22
3.3.4 Sampling Coherence Scenarios	25
4 Results & Evaluation	26
4.1 Experimental Setup	26
4.2 Image Accuracy	27
4.3 Memory Footprint	29
4.3.1 NanoVDB	29
4.3.2 Two-Level Grid	30
4.4 Point-Query Throughput	32
4.4.1 Linear Query	32
4.4.2 Random Query	33
4.5 Rendering Performance	35
4.5.1 Fixed-Step Marching	35
4.5.2 Space-Skipping Impact	36
4.6 Summary	38

5 Conclusion, Limitations and Future Work	39
5.1 Conclusion	39
5.2 Limitations	39
5.3 Future Work	40
Bibliography	41

1 Introduction

1.1 Problem Statement and Motivation

Volumetric datasets are foundational to computer graphics, visual effects, physical simulations, and medical imaging. Unlike standard polygonal meshes, a volume represents continuous properties such as density or temperature across three-dimensional space. In isolation, a single sample point reveals little information because volumetric structures such as clouds, smoke, or fluids only make sense when examining millions or billions of sample values. Evaluating these continuous fields during rendering requires a greater number of signal reconstructions between discrete sample positions. Specifically, the performance of a rendering algorithm is heavily influenced by the way the dataset is stored. This issue introduces the field of different volumetric data structures that minimize the raw memory footprint while also optimizing the performance of data fetches. Simple data structures do not take the underlying *shape* of the data into account, even though many volumetric datasets such as clouds usually feature a high sparsity, meaning that most of the stored values do not actually contribute to the final image. More complex data structures, such as multi-level tree structures may introduce excessive overhead and complexity for simple static scenes. Hierarchical structures such as octrees [Meagher, 1982], OpenVDB [Museth, 2013] and NanoVDB [Museth, 2021] all recursively subdivide the bounding box of a volume into smaller regions and omit *empty* ones. While these methods all fit to the *shape* of the underlying data, they also introduce organizational overhead in the form of indirection layers. Each additional layer of indirection requires resolving memory addresses in steps prior to reading values from the data. On graphics cards, threads execute in parallel groups and rely on fast linear cache reads to stay busy. When sample lookups stall while waiting for pointers to resolve, rendering throughput drops. Additionally, rendering algorithms like ray marching need to step through the data, which results in a ray marcher that steps through empty air most of the time. A volumetric data structure must therefore balance data compression against the cost of querying samples as well as optimize the performance of rendering algorithms like ray marching.

1.2 Research Question and Thesis Scope

This thesis focuses on volumetric clouds (see Figure 1.1) stored as density values across three-dimensional space. All test clouds are taken from the *cloudy* dataset and have a resolution of approximately $400 \times 200 \times 400$ voxels [Leonard et al., 2025]. Compared to animated fluid simulations, this scenario is fairly simple, since the datasets have a fixed shape and do not change over time. They also feature a moderate bounding box occupancy of roughly 20 to 40 percent. Since the clouds are static and do not change over time, the data structures also remain static. To investigate this specific setting, this thesis implements and measures three volumetric data structures on clouds. The data structures will be tested for linear access and random

access as well as for fixed-step ray marching and an optimized space-skipping ray marcher. These are a baseline dense grid, a custom two-level sparse grid built specifically for this thesis, and NanoVDB [Museth, 2021] as an established sparse format.

The central research question focuses on whether the two-level sparse grid can combine the simplicity of the dense grid with the complex structure of NanoVDB in order to eliminate empty space in memory and during ray marching. The second research question follows directly from this comparison. NanoVDB is built for very sparse, huge scenes. This is a substantially different setting than a small cloud with an occupancy of 20%. Therefore, this thesis sets out to answer the question of whether NanoVDB’s architectural advantages still translate into a measurable benefit under these requirements.



Figure 1.1 ccloud_1107 from the cloudy dataset, rendered with fixed-step ray marching over the dense grid.

1.3 Thesis Outline

Chapter 2 introduces volumetric data and the three families of representation considered in this thesis, dense grids, bricked or two-level sparse grids, and the hierarchical VDB and NanoVDB formats. It then covers the traversal algorithms used to render these representations, ray-box intersection, Beer-Lambert absorption and fixed-step ray marching, space skipping with 3D-DDA and HDDA, and trilinear interpolation. The chapter concludes with an examination of the GPU-specific bottlenecks of warp divergence and memory coalescing that motivate the comparisons carried out in later chapters. Next, chapter 3 describes how the dense grid, the two-level sparse grid, its ghost-voxel padded variant, and NanoVDB were implemented as GLSL compute shaders within the framework. This includes the point sampler built for each structure and the three ray marching strategies constructed on top of them. Chapter 4 presents the experimental setup and evaluates the four representations in terms of image accuracy, memory footprint, point-query throughput under coherent and incoherent access, and rendering performance under both fixed-step and space-skipping traversal, on three clouds of increasing occupancy. Finally, chapter 5 answers the research questions, discusses the limitations of the experimental setup, and outlines directions for future work.

2 Background

2.1 Volumetric Data

[Kaufman \[2000\]](#) defines volumetric data as a finite set of discrete samples (x, y, z, v) , where $v \in \mathbb{R}$ represents a scalar value (e.g. a floating-point density) at a three-dimensional spatial coordinate $(x, y, z) \in \mathbb{R}^3$. These samples are commonly arranged on a *uniform grid* (see Figure 2.1), which subdivides the spatial domain into identically sized, axis-aligned cubic cells whose vertices are referred to as *voxels* [[Engel et al., 2006](#)]. To evaluate the volume at arbitrary continuous positions within the domain, intermediate values are reconstructed via trilinear interpolation.

A volumetric dataset is enclosed by a *bounding box*, typically defined by the minimum and maximum coordinates of the grid. In time-dependent physical simulations, both the bounding domain and the underlying scalar field can change dynamically. In contrast, this thesis focuses exclusively on static volumetric datasets defined within a fixed spatial domain. [[Engel et al., 2006](#)]

Designing an efficient data structure for volumetric data involves balancing conflicting objectives. Trilinear interpolation requires fetching eight adjacent voxels per evaluation, demanding high memory throughput and spatial locality to avoid memory bottlenecks. Concurrently, the cubic memory growth of dense representations presents practical scaling limits, especially given that many volumetric datasets exhibit significant spatial sparsity. Consequently, memory footprint, cache locality, and traversal efficiency represent central architectural trade-offs. The following sections evaluate these trade-offs across common representations on uniform grids.

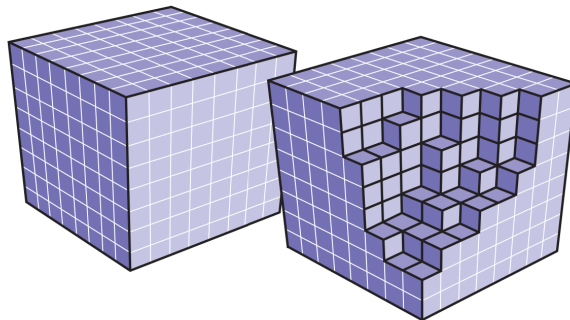


Figure 2.1 Discretization of a continuous volumetric scalar field on a regular uniform grid, showing discrete sample points (voxels) and enclosing cubic cells. Adapted from [Engel et al. \[2006, p. 3\]](#).

2.1.1 Dense Representations

The standard baseline for storing a regular volumetric grid is to allocate contiguous linear memory for the entire spatial domain. Every voxel is stored sequentially in a flat one-dimensional array, regardless of whether it contains active features or empty space. Consequently, this layout is referred to as a *dense* representation.

For a grid of dimensions $W \times H \times D$ (width, height, and depth), exactly $W \cdot H \cdot D$ scalar values must be stored. The resulting memory footprint scales with $\mathcal{O}(W \cdot H \cdot D)$. For a cubic volume where $W = H = D = N$, this simplifies to $\mathcal{O}(N^3)$. Table 2.1 details the resulting memory footprint for 32-bit floating-point data across various resolutions N .

Resolution per Axis (N)	Total Voxels (N^3)	Memory Footprint (GiB)
128	2 097 152	0.01
256	16 777 216	0.06
512	134 217 728	0.50
1024	1 073 741 824	4.00
2048	8 589 934 592	32.00
4096	68 719 476 736	256.00

Table 2.1 Memory footprint of a dense representation using 32-bit floating-point values (4 bytes per voxel) across various resolutions N .

Directly mapping three-dimensional coordinates to one-dimensional memory via strides enables $\mathcal{O}(1)$ random access without pointer indirection. However, this layout exhibits poor spatial locality in memory. As illustrated in Figure 2.2 for a two-dimensional case, adjacent voxels along the x -axis (e.g., voxels 9 and 10) reside contiguously in memory, whereas neighboring voxels along the y -axis (e.g., voxels 3 and 7) are separated by large stride offsets. Since rays traverse the volume at arbitrary angles rather than along axis-aligned paths, this access pattern frequently requires fetching data from distant, non-contiguous regions in VRAM, which limits memory efficiency.

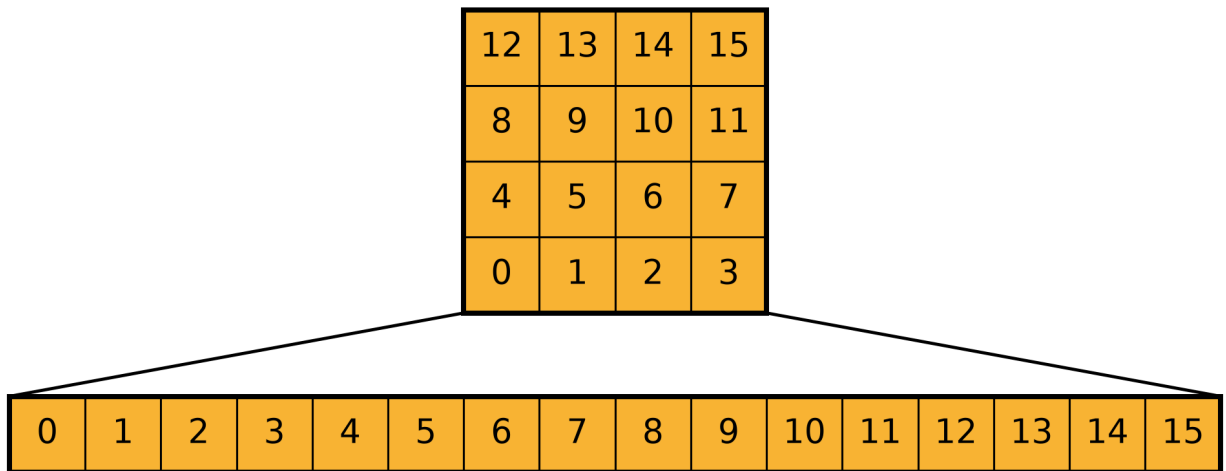


Figure 2.2 Storage layout of a two-dimensional dense grid in a flat one-dimensional array. Voxels adjacent along x (e.g. 9 and 10) are neighbors in memory, while voxels adjacent along y (e.g. 3 and 7) are separated by a full row stride.

In addition to memory locality issues, dense grids suffer from redundant storage of empty regions. Since every voxel is explicitly allocated, transparent or empty regions consume equal memory to dense areas. For instance, a sparse medium occupying only 10% of its bounding box has the exact same storage cost as an object occupying 90% of the volume.

2.1.2 Bricked Representations

To overcome the spatial locality and storage limitations of flat dense arrays, the spatial domain can be subdivided into smaller regular sub-volumes. While hierarchical structures such as *octrees* [Meagher, 1982] partition space recursively, a flatter and conceptually simpler approach is uniform block decomposition, commonly referred to as *bricking* [Engel et al., 2006, Parker et al., 1999].

Bricking addresses multiple architectural challenges. In large-scale volume rendering, partitioning data into uniform blocks facilitates out-of-core streaming, as only the sub-volumes relevant to a specific scene must reside in memory. Although out-of-core handling is unnecessary for the datasets evaluated in this work (which remain below resolutions of 500^3), bricking provides a substantial advantage by restructuring the physical memory layout to preserve spatial locality [Parker et al., 1999].

By organizing the volume into cubic bricks of edge length B (containing B^3 voxels), all data points within a block are stored contiguously in memory. Since the spatial extent B of an individual brick is significantly smaller than the total grid dimension N , neighboring voxels along all spatial dimensions remain physically close in linear memory [Parker et al., 1999]. Figure 2.3 illustrates this layout for a two-dimensional domain with $B = 2$, showing how local 2×2 neighborhoods form contiguous segments in linear memory, in contrast to the dense baseline in Figure 2.2.

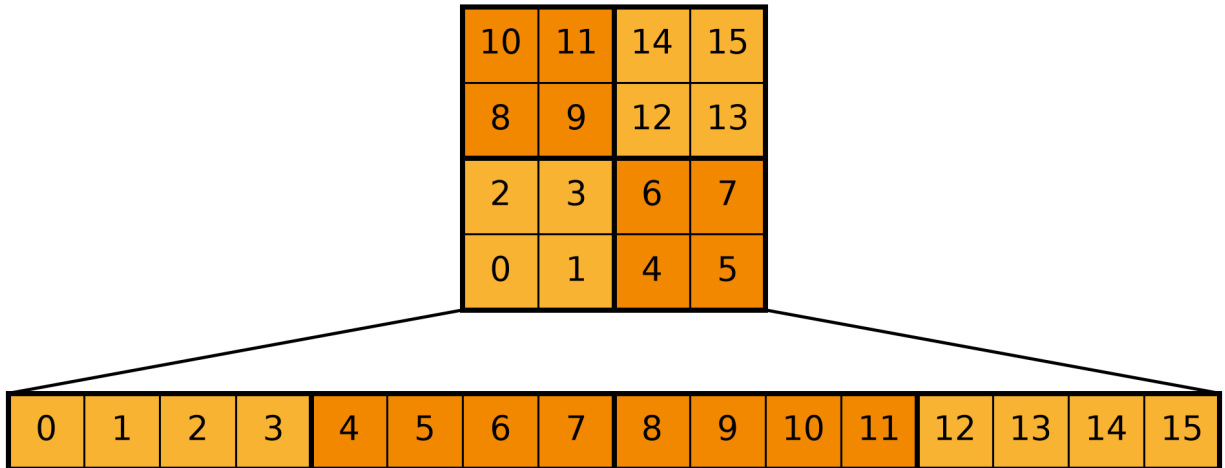


Figure 2.3 Storage layout of a two-dimensional grid partitioned into uniform bricks ($B = 2$), ensuring that local neighborhoods reside contiguously in linear memory.

Accessing a voxel in a bricked volume requires mapping global coordinates to a brick identifier or offset. For a two-level structure with fixed-size bricks, this mapping introduces a single layer of indirection, such as an index table. This indirection incurs a constant $\mathcal{O}(1)$ lookup time, while adding minimal memory overhead proportional to the total number of bricks, $\mathcal{O}((N/B)^3)$. Furthermore, this decoupled indexing

allows completely omitting unallocated or empty bricks, effectively transforming the bricked grid into a memory-efficient sparse representation [Beyer et al., 2015].

2.1.3 OpenVDB

Museth [2013] introduced *VDB*, a hierarchical data structure developed for sparse, time-varying volumetric data, which was later released as the open-source library *OpenVDB*. The acronym *VDB* reflects its core design, a *Volumetric, Dynamic* grid that incorporates architectural principles from B^+ trees. Unlike static sparse grids, VDB is specifically designed for dynamic topologies, enabling efficient runtime insertion and deletion of data during physical simulations. Furthermore, rather than requiring a pre-allocated bounding box, VDB provides an effectively unbounded index space.

While hierarchical spatial subdivision using *octrees* [Meagher, 1982] has long been used in volumetric processing, octrees suffer from deep traversal paths due to their small branching factor of two along each spatial dimension. The design of VDB is directly inspired by the B^+ tree, a self-balancing search tree widely used in relational database management systems and file systems [Comer, 1979]. VDB adapts three primary concepts from B^+ trees:

1. **Large branching factors:** Similar to B^+ trees, VDB nodes utilize high branching factors (for example, internal nodes configured with resolutions such as $32 \times 32 \times 32$ or $16 \times 16 \times 16$).
2. **Shallow tree depth:** Due to these large branching factors, the tree depth remains very shallow, typically three to four levels. This significantly reduces pointer dereferencing and traversal overhead compared to deep octrees.
3. **Height-balanced construction:** All leaf nodes reside at the exact same depth from the root node, ensuring constant-depth traversal paths and enabling efficient bitwise coordinate decoding.

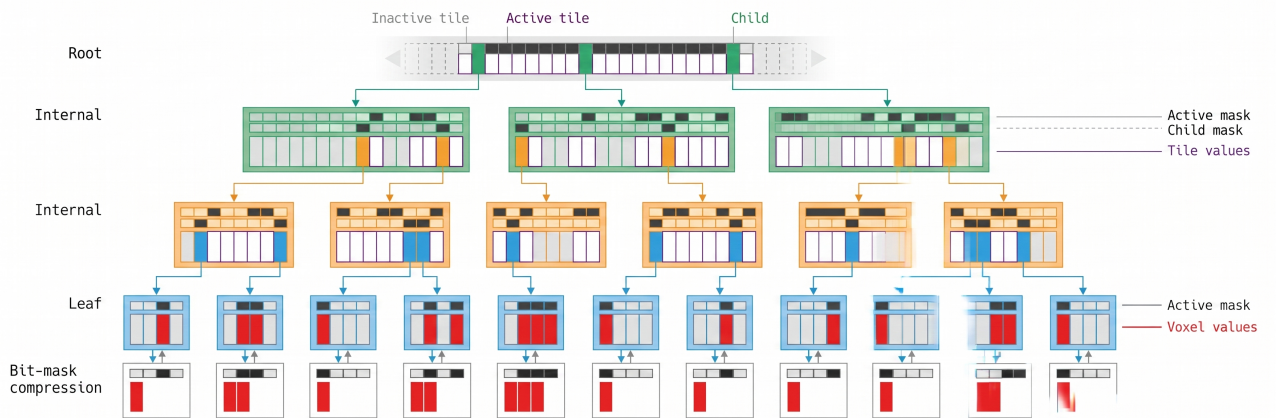


Figure 2.4 Hierarchical structure of a VDB grid with a single root node, showing the root, the internal nodes and the leaf nodes at the base of the tree. Adapted from Museth [2013].

Figure 2.4 illustrates the hierarchy of a VDB grid which is structured into three node types: the root, intermediate internal nodes, and bottom-level leaf nodes. The root defines an unbounded coordinate space, eliminating the need for a pre-allocated bounding box. To support this unbounded domain, the root maintains a hash map that maps three-dimensional coordinate keys to child branches or root-level

background tiles. Each child branch under the root spans a large sub-volume. For instance, in the standard configuration, each branch spans a domain of 4096^3 .

The subsequent tier consists of intermediate internal nodes. In the standard configuration, the first internal node manages a domain of 4096^3 voxels and subdivides it into $32 \times 32 \times 32 = 32,768$ sub-blocks of size 128^3 voxels. For each entry, the node maintains a reference alongside two compact bit-masks:

- **Child mask:** Indicates whether the corresponding spatial sub-region is subdivided. If the bit is 1, the slot stores a pointer to a child node covering that 128^3 region. If the bit is 0, the region is not subdivided and instead stores a single uniform *tile value* representing the entire 128^3 volume.
- **Active mask:** When an entry is stored as an undivided tile, this bit indicates whether the tile is active (representing foreground content, such as a constant interior value) or inactive (representing empty background space).

The next internal level operates on the same architectural principle: each second-level internal node manages a 128^3 sub-volume and branches by $16 \times 16 \times 16 = 4096$ slots into blocks of size 8^3 voxels.

At the base of the hierarchy reside the leaf nodes at a fixed depth. In the standard configuration, each leaf node manages a $8 \times 8 \times 8 = 512$ voxel block. A leaf node stores a contiguous array of 512 raw voxel values alongside a 512-bit active mask that explicitly marks each individual voxel as active or inactive.

Figure 2.4 illustrates OpenVDB as a tree hierarchical structure. A more conceptual illustration can be found in Figure 2.5.

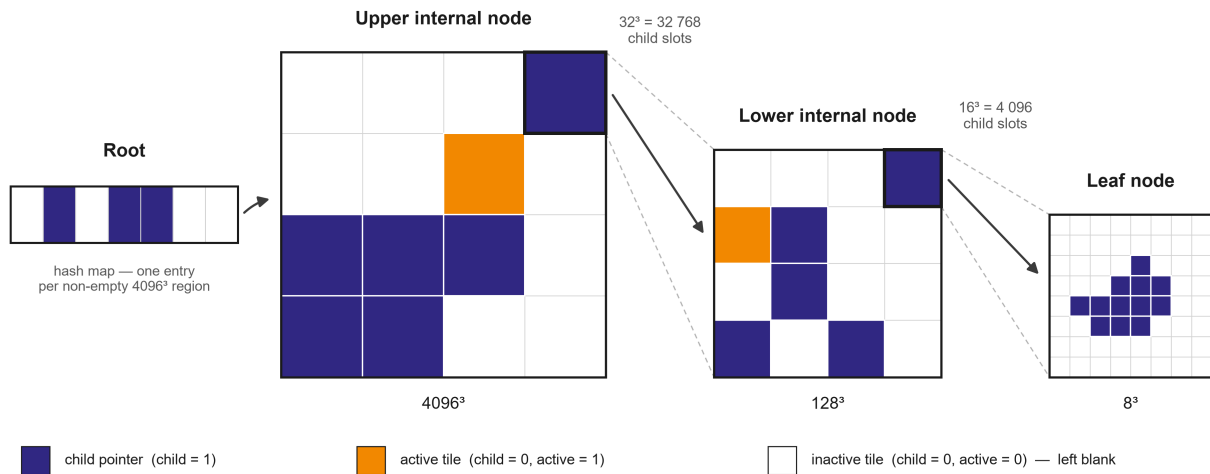


Figure 2.5 Recursive spatial subdivision of a VDB grid in two dimensions, illustrating how root, internal and leaf regions nest within one another.

Unlike dense grids, the memory footprint of a VDB grid cannot be expressed as a simple closed-form function of the bounding box dimensions, as it depends directly on the topological distribution and sparsity of active voxels. Museth [2013] demonstrated memory footprint by using a polygonal model from *How to Train Your Dragon* with an effective bounding box of $7897 \times 1504 \times 5774$ voxels. While an equivalent dense grid would have required approximately 250 GiB of memory, the sparse VDB representation consumed only 1 GiB, achieving a 250 $\times$ compression in storage.

In addition to compact memory usage, VDB is built for fast random access. In standard spatial hierarchies such as octrees, access queries exhibit $\mathcal{O}(\log N)$ time complexity, where traversal depth grows with spatial resolution and results in significant pointer-chasing overhead. In VDB, the worst-case time complexity of a random query is strictly bounded by the tree depth D . Since D is small and fixed (typically $D \in \{3, 4\}$), worst-case traversal executes in $\mathcal{O}(1)$ constant time.

While the standard OpenVDB library is built for dynamic simulations on CPUs, its complex pointer structures and memory allocations make it difficult to run directly on GPUs. To solve this, [Museth \[2021\]](#) introduced *NanoVDB*, a version of the VDB data structure tailored specifically for graphics hardware. The core idea of NanoVDB is to flatten the entire tree into a single, contiguous memory buffer. Instead of using raw memory pointers to connect parent nodes to child nodes, it uses relative byte offsets within this single buffer. This allows the entire volume grid to be copied directly from CPU memory to GPU memory using a simple copy without needing any address translation or reconstruction. To achieve maximum rendering speed on the GPU, NanoVDB is designed as a read-only data structure. The volumetric grid is typically constructed and updated on the CPU using OpenVDB and then converted into a compact NanoVDB buffer for the GPU. Additionally, NanoVDB implements the root level as a linear array rather than a hash map. This naturally follows from the linearity needed for a GPU specific data structure.

2.2 Traversals

The previous section introduced three data structures for volumetric data and evaluated their memory footprints and theoretical access times. In practice, overall rendering performance also depends heavily on the traversal and sampling algorithms used to query these structures. A popular direct volume rendering technique relies on ray casting, where view rays are cast from the camera into the volume domain. A viewing ray is defined parametrically as:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \geq 0 \quad (2.1)$$

where $\mathbf{o} \in \mathbb{R}^3$ denotes the ray origin (the camera position), $\mathbf{d} \in \mathbb{R}^3$ is the normalized view direction vector, and t represents the distance traveled along the ray. This section examines the techniques required for traversing $\mathbf{r}(t)$ through a volumetric bounding domain and evaluating samples along its path.

2.2.1 Ray Bounding Box Intersection

Before sampling volumetric data, a ray must be tested for intersection against the global domain boundaries. The spatial domain is typically defined as an axis-aligned bounding box (AABB). A standard approach to compute the entry and exit points of a ray traversing an AABB is the *slab method* [[Kay and Kajiya, 1986](#)]. While early work by [Glassner \[1984\]](#) also employed the slab method for hierarchical space-skipping in octrees, this thesis focuses on *Digital Differential Analyzers (3D-DDA)* [[Amanatides and Woo, 1987](#)] for spatial acceleration, restricting the use of the slab method strictly to global boundary clipping.

Given an AABB defined by its two corner points $\mathbf{p}_{\min}$ and $\mathbf{p}_{\max}$, the ray parameter intervals for each coordinate axis $i \in \{x, y, z\}$ are computed by intersecting the ray with the corresponding slab planes (see Figure 2.6):

$$t_{i,\min} = \min\left(\frac{p_{\min,i} - o_i}{d_i}, \frac{p_{\max,i} - o_i}{d_i}\right), \quad t_{i,\max} = \max\left(\frac{p_{\min,i} - o_i}{d_i}, \frac{p_{\max,i} - o_i}{d_i}\right) \quad (2.2)$$

The global entry distance t_{entry} and exit distance t_{exit} are then given by the latest entry and earliest exit across all three axes:

$$t_{\text{entry}} = \max(t_{x,\min}, t_{y,\min}, t_{z,\min}), \quad t_{\text{exit}} = \min(t_{x,\max}, t_{y,\max}, t_{z,\max}) \quad (2.3)$$

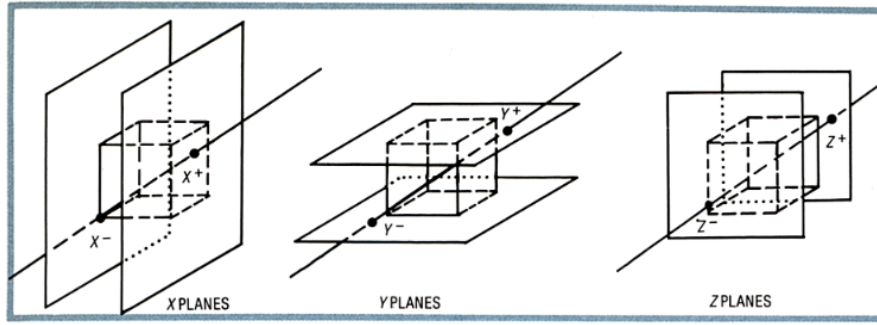


Figure 2.6 Slabs across three dimensions, showing entry and exit points. Adapted from Glassner [1984]

A valid ray–box intersection occurs if and only if $t_{\text{entry}} \leq t_{\text{exit}}$ and $t_{\text{exit}} \geq 0$. In practice, the slab method evaluates this test and either reports a miss if the ray bypasses the volume, or returns the entry and exit interval $[t_{\text{entry}}, t_{\text{exit}}]$. If an intersection is found, ray marching only needs to sample within this interval, bypassing all unallocated empty space outside the bounding box.

2.2.2 Absorption-Only Ray Marching

To visualize scalar density fields, Max [1995] described the absorption-only optical model. In this model, the volume is assumed to consist of perfectly absorbing particles that occlude background light without scattering or emitting radiation.

Light attenuation along a viewing ray path of length D is governed by the differential equation:

$$\frac{dI}{ds} = -\tau(s)I(s) \quad (2.4)$$

where $\tau(s)$ represents the extinction coefficient, proportional to the local particle density. Solving this differential equation yields the continuous Beer-Lambert law:

$$I(D) = I_0 \exp\left(-\int_0^D \tau(s) ds\right) \quad (2.5)$$

where I_0 denotes the background illumination entering the volume, and the integral term represents the accumulated optical depth along the ray.

For numerical volume rendering, the continuous optical depth integral cannot be evaluated analytically. Instead, the ray path across the volume domain $[0, D]$ is divided into N uniform segments of length Δs . The total optical depth is then approximated via a discrete Riemann sum:

$$\tau_{\text{total}} \approx \sum_{i=1}^N \tau(s_i) \Delta s \quad (2.6)$$

As noted by Max [1995], approximating the integral directly provides a significant computational advantage: the optical depth can be accumulated through simple summations along the ray, requiring only a single exponential evaluation at the end of the trajectory to determine final transmittance:

$$T(D) = \exp(-\tau_{\text{total}}) \quad (2.7)$$

2.2.3 Space-Skipping via Digital Differential Analyzers

While sparse data structures avoid storing empty voxels in memory, ray marching algorithms must also actively leverage this sparsity during rendering. Marching a ray through empty space using fine, uniform step sizes leads to computational overhead without contributing to the final accumulated transmittance. Consequently, efficient volume renderers employ space-skipping, advancing rays through unoccupied regions in large intervals rather than fixed steps.

A foundational method for determining these traversal intervals is the *three-dimensional digital differential analyzer (3D-DDA)*, introduced by Amanatides and Woo [1987]. Although originally proposed for exact voxel-by-voxel grid traversal, the algorithm generalizes naturally to macro-cell structures such as bricked volumes. By treating each non-empty brick as a single cell in a coarse occupancy grid, 3D-DDA computes the exact entry and exit points along the ray, allowing empty bricks to be bypassed in a single leap. Figure 2.7 illustrates the core principle in a two-dimensional setting.

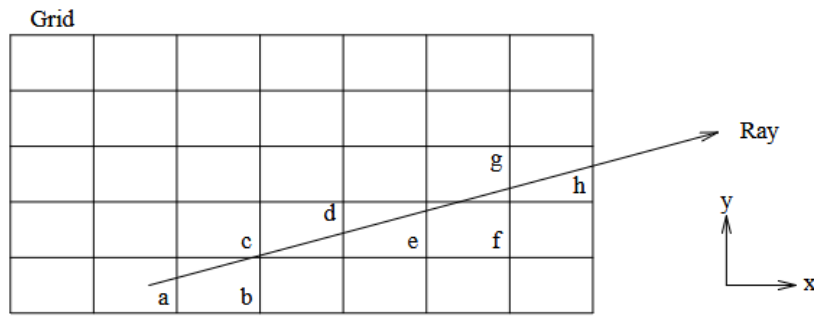


Figure 1

Figure 2.7 Problem statement of the 3D-DDA simplified to two dimensions: the algorithm identifies and visits all intersected cells (a through h) in strict front-to-back sequence [Amanatides and Woo, 1987].

The algorithm operates incrementally along the ray's trajectory $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$, ensuring that all intersected cells are traversed in front-to-back order. For each dimension $i \in \{x, y, z\}$, the traversal state maintains the parametric ray distance t_i to the next cell boundary along that axis and the constant parametric step length Δt_i required to traverse an entire cell width. In each step, the algorithm identifies the axis with the minimal t_i , increments the cell index along that axis, and advances t_i by Δt_i . Since each iteration relies

exclusively on scalar additions and comparisons, the traversal is computationally lightweight and avoids redundant ray-plane intersections.

To extend this concept to multi-level spatial structures, [Museth \[2014\]](#) proposed the *Hierarchical Digital Differential Analyzer (HDDA)*. Unlike standard 3D-DDA, which operates on a single flat grid resolution, HDDA dynamically adjusts its step sizes by ascending and descending through the levels of a hierarchical volume tree, such as OpenVDB. When entering an empty region, HDDA moves up the hierarchy to step across entire large parent nodes at once. This hierarchical ray-marching model was subsequently ported to GPU architectures in NanoVDB [\[Museth, 2021\]](#).

2.2.4 Trilinear Interpolation

Trilinear interpolation is an extension of linear interpolation to a three-dimensional regular grid, allowing continuous sampling at arbitrary coordinates within a cell bounded by eight neighboring voxels. Conceptually, it is performed as a hierarchical sequence of seven one-dimensional linear interpolations, or equivalently, as two bilinear interpolations followed by a final linear step.

Given a normalized local coordinate $(x_d, y_d, z_d) \in [0, 1]^3$ within the voxel cell and following the notation in [Figure 2.8](#), the interpolated value c is computed sequentially along the coordinate axes:

$$\begin{aligned}
 c_{00} &= c_{000}(1 - x_d) + c_{100}x_d \\
 c_{01} &= c_{001}(1 - x_d) + c_{101}x_d \\
 c_{10} &= c_{010}(1 - x_d) + c_{110}x_d \\
 c_{11} &= c_{011}(1 - x_d) + c_{111}x_d \\
 c_0 &= c_{00}(1 - y_d) + c_{10}y_d \\
 c_1 &= c_{01}(1 - y_d) + c_{11}y_d \\
 c &= c_0(1 - z_d) + c_1z_d
 \end{aligned}$$

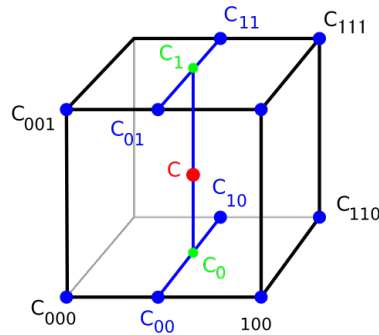


Figure 2.8 Trilinear interpolation within a cubic grid cell, showing the eight corner voxels and the intermediate points used in the sequence of linear interpolations. Adapted from [Wikipedia contributors \[2024\]](#).

This method applies to any scalar or vector quantity defined at the grid vertices. In the context of volume rendering, the corner vertices typically store discrete density or color values, and trilinear interpolation reconstructs a smooth, continuous scalar field across the domain.

2.3 GPU-Specific Bottlenecks

The previous sections introduced three volumetric data structures and the algorithmic components required for ray marching. While the memory footprint is an important metric, rendering performance is equally crucial. Due to their structural differences, these data structures exhibit distinct performance characteristics during traversal and sampling. To understand these differences, this section examines common GPU-specific bottlenecks, outlines the relevant aspects of modern GPU execution models, and discusses why ray marching maps efficiently to parallel hardware.

Modern GPUs achieve high computational throughput by executing thousands of threads in parallel. In NVIDIA architectures, threads within a thread block are grouped into execution units of 32 threads named *warps*. A warp serves as the fundamental unit of scheduling and execution on the GPU, operating under the Single-Instruction Multiple-Threads (SIMT) paradigm. This model is particularly well-suited for ray marching because computing the color for a given pixel is an independent process, allowing individual pixels to be evaluated concurrently by separate threads. While all active threads within a warp synchronously receive the same instructions, individual threads can take different execution paths depending on runtime conditions. When threads within a single warp branch into different paths, *warp divergence* occurs. In the scope of volumetric ray marching, common causes for this are early ray termination and space-skipping. To handle divergent control flow, the GPU executes one path while masking off the inactive lanes, and vice versa. Figure 2.9 illustrates warp divergence caused by a conditional statement. Since inactive lanes result in underutilized execution units, minimizing divergence is essential for maximizing GPU performance. [NVIDIA Corporation, 2026a]

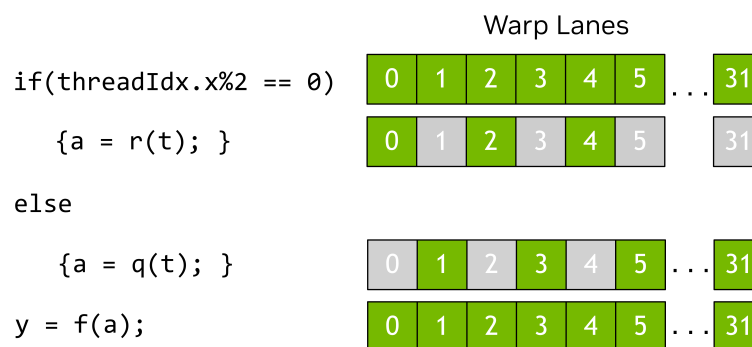


Figure 2.9 Warp divergence caused by an if-else statement [NVIDIA Corporation, 2026a].

In addition to warp divergence, memory access patterns heavily influence rendering performance. Fetching data from global video memory (VRAM) is slow, taking approximately 800 clock cycles. When data is loaded, it is stored in the L1 and L2 caches, where latency drops to around 35 and 350 cycles, respectively (NVIDIA Blackwell architecture). Furthermore, fetching a 32-bit floating-point number loads an entire 128-byte cache line into the caches. When adjacent threads in a warp access contiguous addresses simultaneously, the hardware *coalesces* these requests into a single memory transaction. High spatial

and temporal memory locality is therefore essential to maximize cache reuse and avoid redundant stalls. [\[Jarmusch et al., 2025\]](#)

Although VRAM accesses have high latency, GPUs rely on latency hiding to maintain throughput. When one warp stalls waiting for memory, the warp scheduler switches to execute another ready warp. However, latency hiding becomes significantly more challenging when memory accesses are mutually dependent, such as when a first read retrieves an offset or pointer needed to address a second read. These serialized dependencies lead to extended stalls that are much harder to hide. This problem is particularly relevant for data structures with indirection layers, because the data can only be read once the offset is retrieved from the indirection layer. [\[Buck, 2005\]](#)

3 Implementation

3.1 Architectural Overview

All volumetric representations and benchmark pipelines in this thesis are implemented within the Python framework *Rendervous* (*rdv*) [Leonard, 2024a], which interfaces with the Vulkan API via the *vulky* library [Leonard, 2024b]. Volumetric datasets and camera ray parameters are allocated and managed directly in GPU memory as *PyTorch* tensors [Paszke et al., 2019]. The GPU pipeline is executed using modular compute shaders written in GLSL [Kessenich et al., 2019]. The code is publicly available on GitHub.¹

The rendering pipeline separates ray traversal from data reconstruction. Traversal is implemented via three distinct ray marching strategies: a fixed-step baseline, a 3D-DDA traversal for the two-level sparse grid, and a HDDA traversal for NanoVDB. Data reconstruction is handled by dedicated sampling modules implemented for each of the three data structures. Given a three-dimensional coordinate, each module returns the reconstructed scalar density via trilinear interpolation over the eight surrounding voxels retrieved from the corresponding data structure. The fixed-step baseline calls this sampling function at every step along the ray, independent of the underlying structure. The two space-skipping traversals reuse the same per-structure sampling module for density reconstruction, but additionally query each structure’s own occupancy information to determine how far a step can advance through empty space before fixed steps are needed again. The overall architecture is illustrated in Figure 3.1.

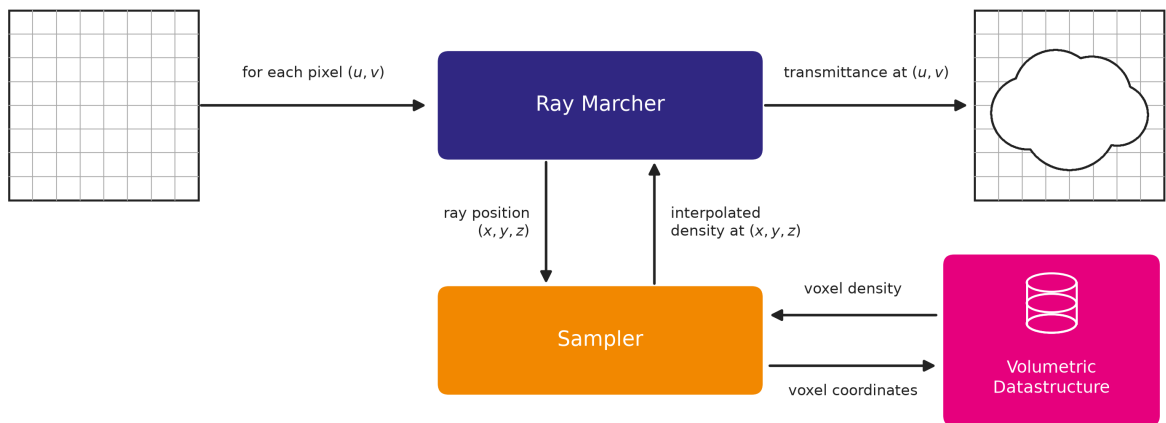


Figure 3.1 Data flow of the shared rendering pipeline. For each pixel (u, v) the ray marcher steps along the view ray and passes every ray position (x, y, z) to the sampler, which reconstructs the density at that position by trilinear interpolation over the eight surrounding voxels read from the volumetric data structure and returns it to the marcher. The marcher accumulates optical depth along the ray and outputs the transmittance at (u, v) .

¹<https://github.com/simontheuseful/volumetric-grid-benchmark>

The following section details the concrete implementation and optimization strategies using GLSL-Shader code. To maintain focus on the core algorithms, organizational framework overhead will be ignored (e.g. Rendervous boilerplate).

3.2 Point Sampler Implementations

The first implementations will be shown in form of the sampler functions. Conceptually, each sampler is a function:

$$f: \mathbb{R}^3 \rightarrow \mathbb{R}, \quad (x, y, z) \mapsto d, \quad (3.1)$$

where (x, y, z) denotes a continuous coordinate along the viewing ray, and d represents the reconstructed density. Evaluating f requires fetching eight neighboring voxel values from the underlying data structure and performing trilinear interpolation. Since the memory layout and addressing logic differ across the three data structures, each sampler is discussed and implemented separately.

3.2.1 Dense Grid Sampler

The baseline dense grid sampler stores scalar densities as 32-bit floating-point values in a contiguous dense tensor of shape $(D, H, W, 1)$ in row-major order, with width being the fastest-varying dimension. Each entry in the tensor represents the density value at a corresponding grid point.

The ray marcher hands sample points in normalized device coordinates $x \in [-1, 1]^3$ over to the sampler. The sampler maps these coordinates into continuous voxel space, yielding $(x, y, z) \in [0, W - 1] \times [0, H - 1] \times [0, D - 1]$. In this convention, (x, y, z) corresponds to the spatial dimensions (W, H, D) , matching the memory layout where x is the fastest-varying coordinate.

The voxel coordinate (x, y, z) directly identifies the enclosing cell. The lower-corner index of the cell is obtained by flooring each component, $v_{000} = (x_0, y_0, z_0) = (\lfloor x \rfloor, \lfloor y \rfloor, \lfloor z \rfloor)$, with the upper corner given by $v_{111} = (x_0 + 1, y_0 + 1, z_0 + 1)$. Together, these indices define the eight corner voxels used for trilinear interpolation. When a sample coordinate falls exactly on the boundary of the grid, both the lower and upper voxel coordinates are clamped to the valid index range of the tensor to avoid out-of-bounds access.

Linear memory addresses are computed using per-axis byte strides. Since each voxel occupies 4 bytes, advancing one step along the width axis increases the memory address by $s_x = 4$ bytes, along the height axis by $s_y = 4 \cdot W$ bytes, and along the depth axis by $s_z = 4 \cdot W \cdot H$ bytes. Given the GPU base address returned by `load_tensor()`, the memory address of voxel v_{000} is computed as:

$$\text{addr}(v_{000}) = \text{load_tensor}() + x_0 s_x + y_0 s_y + z_0 s_z. \quad (3.2)$$

If a neighboring voxel falls outside the volume, its stride is simply set to zero instead of being applied, so the address collapses back onto the current voxel. This keeps every read inside the tensor and makes the interpolation reduce to the boundary voxels own value.

Rather than re-evaluating the full multiply-add expression independently for each of the eight cell corners, the implementation reuses the precomputed strides s_x, s_y, s_z . The addresses of the remaining seven

corners are derived from $\text{addr}(v_{000})$ via incremental integer additions of the corresponding strides. Finally, trilinear interpolation is performed across the eight fetched density values using seven sequential linear interpolations.

3.2.2 Two-Level Grid Sampler

The two-level sparse grid partitions the volume into cubic bricks of fixed side length B , referred to as *blocks*. The block size B is constant for a given grid and constrained to a power of two ($B = 2^n, n \in \mathbb{N}$), which allows the block index and local voxel index to be recovered from a global voxel index using bit-shifting and masking instead of integer division. Before any voxel value can be read, an indirection layer is required. Rather than storing density values for every block position, a dense array (referred to as *macro array*) stores, for every possible block position, a single 32-bit integer index offset.

Although the macro array covers every block position and is therefore itself dense, its memory footprint is small compared to a full-resolution dense grid, since a single macro array entry replaces an entire $B \times B \times B$ region of voxels. Given a grid dimension of (W, H, D) and block size B , the macro array contains $\frac{W}{B} \cdot \frac{H}{B} \cdot \frac{D}{B}$ elements. For a common block size of $B = 32$ and a bounding box of 512^3 voxels, this results in a macro array of only $16^3 = 4096$ elements, or 16 KiB at 4 bytes per index, which is negligible compared to the roughly 512 MiB required by an equivalent full dense grid of 32-bit floating-point values.

The second layer is the actual sparse array holding the voxel data. This array will be referred to as the *micro array*. Its memory footprint depends on the number of blocks that are actually allocated. A block is unallocated (or *inactive*) if all of its voxels store a density value of zero. It is *active* if there exists at least one voxel with a non zero density. While inactive blocks still require an entry in the macro array, they do not contribute to the final image and are therefore omitted from the micro array.

Given K active blocks, the micro array stores each one of them with B^3 entries. This results in a closed-form formula for the total memory footprint of the two-level grid:

$$M_{\text{total}} = \underbrace{\left(\frac{W}{B} \cdot \frac{H}{B} \cdot \frac{D}{B} \right) \cdot 4}_{M_{\text{macro}}} + \underbrace{K \cdot B^3 \cdot 4}_{M_{\text{micro}}} \quad (3.3)$$

Locating a voxel begins exactly as in the dense grid. The sampling position in $[-1, 1]^3$ is mapped to voxel coordinates and floored to the integer index of the lower corner voxel of the cell, then clamped to the grid bounds. This integer index is then split into two parts. Since the block size B is a power of two, the split is a bit shift and a bit mask rather than a division and modulo operation. The higher bits give the macro block coordinate $\mathbf{b}$, the position of the block in the macro array, and the lower bits give the intra-block coordinate $\mathbf{u}$, the position of the voxel within that block, with each component of $\mathbf{u}$ in the range 0 to $B - 1$.

The macro array itself is a dense array with one 32-bit integer per macro cell, and it is indexed exactly like the dense grid. A byte offset is formed from $\mathbf{b}$ with per-axis strides, the innermost axis having a stride of 4 bytes, and added to the macro array's base address. The value read from that address is not a memory offset but rather a block slot number k , that identifies *which* stored block holds the data for $\mathbf{b}$. Active blocks are numbered from 0 upwards and any empty macro cell stores $k = -1$.

The micro array stores the active blocks back to back, each a contiguous $B \times B \times B$ region of voxels. Turning k into an address is therefore a multiplication by the fixed byte size of one block, $B^3 \cdot 4$ bytes, which lands on the first voxel of block k . The offset of $\mathbf{u}$ within the block, again formed from per-axis strides, is added to this, giving the final address of the voxel.

Locating one voxel in the two-level grid thus always costs one dependent memory read more than the dense grid. The macro array must be read to obtain k before the address in the micro array, and hence the density value itself, can be computed. Since a cell can span a block boundary, v_{000} and v_{111} do not necessarily resolve to the same k , each of the eight densities have to be fetched independently. Two further optimizations were implemented to address this overhead.

The first optimization concerns how an empty block is identified. One option is to store -1 inside the macro array for an empty block. Storing -1 requires an if-statement to check whether the returned index is an actual block index or the defined empty code. Such a branch can cause warp divergence whenever some threads in a warp read from an active block while others read from an empty one. To avoid this branch entirely, the macro array instead stores $k = 0$ for an empty block. A fixed region of size $B^3 \cdot 4$ bytes, filled entirely with zero density, is permanently stored at the very beginning of the micro array. Therefore any fetch resolving to this block always yields zero density without ever needing to be checked. The exact memory footprint formula is then given by:

$$M_{\text{total,zero-block}} = \underbrace{\left(\frac{W}{B} \cdot \frac{H}{B} \cdot \frac{D}{B} \right) \cdot 4}_{M_{\text{macro}}} + \underbrace{(K \cdot B^3 \cdot 4)}_{M_{\text{micro}}} + \underbrace{(B^3 \cdot 4)}_{\text{Zero-Block}} \quad (3.4)$$

For a common block size of 32 this adds only a negligible memory overhead of 128 KiB, while removing the branch entirely.

Second, up to eight independent macro array lookups may still be required per trilinear interpolation, since v_{000} through v_{111} can resolve to different macro block coordinates $\mathbf{b}$ whenever the cell straddles a block boundary. Unlike the purely arithmetic addressing of the dense grid, each lookup is a dependent memory fetch, since the micro array address cannot be computed until the macro array read has finished. To avoid repeating this dependent read up to eight times per sample, every stored block is padded with one additional layer of ghost-voxels (see Figure 3.2), copied from its immediate $+x$, $+y$, and $+z$ neighboring block, growing it from B^3 to $(B+1)^3$ voxels. Since the intra-block coordinate $\mathbf{u}$ always lies in $[0, B-1]$, the corresponding upper corner $\mathbf{u} + 1$ is now guaranteed to still fall within this same padded block, removing the need to ever resolve a second macro block coordinate. With this additional layer, only a single macro array read, for $\mathbf{b}$ derived from v_{000} , is required per sample. The remaining seven corner addresses are then obtained by pure pointer arithmetic from one base address, exactly as in the dense grid. This removes seven of the eight dependent macro array reads per trilinear sample, at the cost of some memory overhead in form of redundancy.

The cost of this is a larger micro array, since each stored block now occupies $(B+1)^3$ entries instead of B^3 . The micro array size becomes

$$M_{\text{micro,padded}} = K \cdot (B+1)^3 \cdot 4. \quad (3.5)$$

Relative to the unpadded micro array M_{micro} defined in Equation 3.3, the size grows by the factor

$$\frac{M_{\text{micro,padded}}}{M_{\text{micro}}} = \left(\frac{B+1}{B} \right)^3, \quad (3.6)$$

which decreases towards one as the block size increases. Table 3.1 lists this factor for different block sizes B . For small blocks the relative overhead is large, reaching a factor of 3.375 at $B = 2$, whereas at $B = 32$ it amounts to only about ten percent.

B	B^3	$(B+1)^3$	$((B+1)/B)^3$	Overhead
2	8	27	3,375	$\approx 238\%$
4	64	125	1,953	$\approx 95\%$
8	512	729	1,424	$\approx 42\%$
16	4096	4913	1,199	$\approx 20\%$
32	32768	35937	1,097	$\approx 10\%$

Table 3.1 Voxel count of an unpadded block, of a padded block, their ratio, and the resulting relative memory overhead.

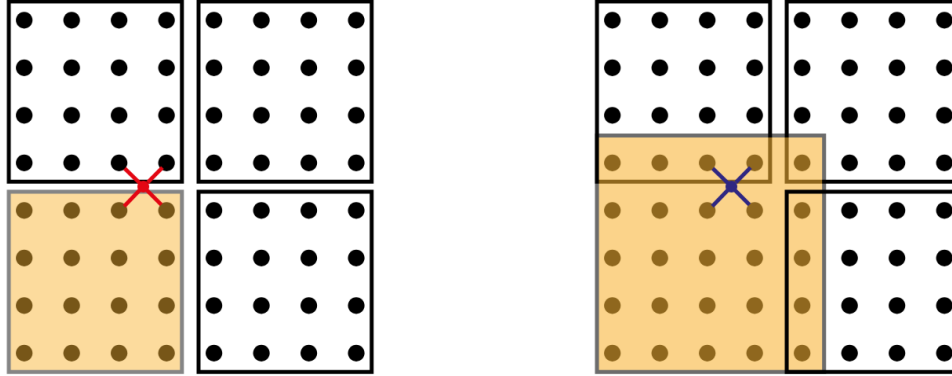


Figure 3.2 Non-padded blocks (left) versus ghost-voxel padded blocks (right) in two dimensions, showing the extra border layer copied from the neighboring blocks.

3.2.3 NanoVDB Sampler

Unlike the two-level grid, NanoVDB is not a structure designed as part of this thesis but rather an established sparse volume format. NanoVDB defines its own binary file format, `.nvdb`. In this thesis, all volume data is exported to this format using NVIDIA's *fVDB* library [Williams et al., 2024]. NanoVDB itself is a C++ library intended for host and CUDA code and cannot be compiled directly into a GLSL compute shader. NVIDIA addresses this with *PNanoVDB*, a single, dependency-free header that reimplements the same tree-traversal logic in a restricted subset of C and compiles largely unchanged as GLSL, HLSL, C++, or CUDA code [Museth and Academy Software Foundation, 2024].

Rather than assuming a specific memory API, PNanoVDB accesses grid data through an abstract buffer type `pnanovdb_buf_t` together with custom read callbacks. Since NanoVDB grids are read-only at render time, only byte-level read operations must be implemented. Code 3.1 implements this interface by bridging PNanoVDB directly to the renderer’s GPU buffer pointers.

Code 3.1 Custom buffer backend bridging PNanoVDB to GPU pointer reads.

```

1 struct pnanovdb_buf_t { GPUPtr base; };
2
3 pnanovdb_buf_t pnanovdb_make_buf(GPUPtr base)
4 {
5     pnanovdb_buf_t buf;
6     buf.base = base;
7     return buf;
8 }
9
10 uint pnanovdb_buf_read_uint32(pnanovdb_buf_t buf, uint byte_offset)
11 {
12     return uint_ptr(buf.base + byte_offset).data[0];
13 }

```

Code 3.2 shows how the shader establishes the tree handles and prepares a read accessor for point sampling.

Code 3.2 Shader initialization of the PNanoVDB hierarchy and read accessor.

```

1 pnanovdb_buf_t buf = pnanovdb_make_buf(load_tensor(parameters.nvdb_data));
2 pnanovdb_grid_handle_t grid_handle = pnanovdb_grid_handle_t(null());
3 pnanovdb_grid_type_t grid_type = PNANOVDB_GRID_TYPE_FLOAT;
4
5 pnanovdb_tree_handle_t tree = pnanovdb_grid_get_tree(buf, grid_handle);
6 pnanovdb_root_handle_t root = pnanovdb_tree_get_root(buf, tree);
7
8 pnanovdb_readaccessor_t acc;
9 pnanovdb_readaccessor_init(acc, root);

```

This initialization sets up the following handles and objects:

buf The buffer object that every subsequent PNanoVDB call reads through. It wraps the `GPUPtr` returned by `load_tensor(parameters.nvdb_data)`, which is the GPU address of the header-stripped `.nvdb` tensor.

grid_handle A byte offset relative to `buf`’s base address. It is set to zero because the file’s outer container header was already removed, so the `GridData` structure begins at the first byte of the buffer.

grid_type The grid value type, fixed to `PNANOVDB_GRID_TYPE_FLOAT` because every volume used in this thesis is a single-channel 32-bit floating-point density grid. Later PNanoVDB calls use it to determine the byte size and layout of a stored voxel.

tree, root Byte addresses of the Tree and Root structures, obtained from `pnanovdb_grid_get_tree` and `pnanovdb_tree_get_root`. Both structures follow the grid header in a fixed layout, so these calls are pure offset arithmetic rather than a traversal. `root` is the linear array structure introduced earlier, containing 4096^3 regions, and is the starting point for the per-coordinate descent performed by a read function.

acc NanoVDB's *read accessor* (`pnanovdb_readaccessor_t`), initialized from `root`. It caches, for each non-root level of the tree (leaf, lower internal, and upper internal), a handle to the node that resolved the previous query. For a new coordinate it resumes traversal from the deepest cached node that still contains it, rather than redescending from the root, which amortizes traversal cost across spatially coherent queries.

As in the previous implementations, the ray coordinate is transformed into voxel index space and clamped to prevent out-of-bounds access at the grid boundary. The resulting voxel coordinates v_{000} and v_{111} again denote the two extreme corners of the enclosing cell, from which the remaining six corner coordinates are obtained by combining their components. Code 3.3 shows the function used to retrieve the density value at each of the eight corners, which are then trilinearly interpolated to yield the final density value.

Code 3.3 Returns the density for a given voxel at coordinates (i,j,k).

```

1 float nanovdb_sample(pnanovdb_buf_t buf, pnanovdb_grid_type_t grid_type,
2   inout pnanovdb_readaccessor_t acc, ivec3 ijk) {
3   pnanovdb_address_t address = pnanovdb_readaccessor_get_value_address(
4     grid_type, buf, acc, ijk);
5   return pnanovdb_read_float(buf, address);
6 }
```

3.3 Traversal & Sampling Strategies

The previous section presented three separate sampler implementations, one for each data structure. As illustrated in Figure 3.1, these samplers reconstruct an interpolated density value at a given ray position (x, y, z) . This section presents the implementation of the *ray marcher*, which advances along the ray and repeatedly invokes the corresponding sampler to accumulate density. Three ray marching strategies are implemented: a fixed-step marcher, a 3D-DDA marcher for the two-level sparse grid, and an HDDA marcher for NanoVDB.

3.3.1 Ray-AABB Intersection

Since every ray in this renderer is intersected against the same symmetric box $[-1, 1]^3$ rather than an arbitrary one, the general slab-based ray-box test can be simplified. For an arbitrary box with bounds $p_{\min}, p_{\max}$, the two boundary crossings per axis are

$$C_{\min} = \frac{p_{\min} - o}{w}, \quad C_{\max} = \frac{p_{\max} - o}{w}, \quad (3.7)$$

where o denotes the ray origin and w its direction. Determining which of $C_{\min}$, $C_{\max}$ is the entry and which is the exit requires an explicit min/max comparison, since the ordering depends on the sign of w .

Since the box is symmetric ($p_{\min} = -1$, $p_{\max} = 1$), both crossings can instead be expressed as a shared center offset by a shared distance,

$$C_{\min} = x_c - d, \quad C_{\max} = x_c + d, \quad x_c = -\frac{o}{w}, \quad d = \frac{1}{w}. \quad (3.8)$$

Replacing d with $s = |d|$ removes the sign ambiguity entirely, since $s \geq 0$, $x_c - s$ is guaranteed to be the smaller crossing and $x_c + s$ the larger one, regardless of the sign of w . This reduces the per-axis entry/exit determination from an explicit comparison to a single `abs()` call, and additionally collapses the two full divisions $C_{\min}$, $C_{\max}$ into a single $\frac{1}{w}$. The ray's entry and exit times are then obtained as the latest per-axis entry (max across axes) and the earliest per-axis exit (min across axes).

Intersections against the unit box are handled by the shader function `ray_box_intersection(x, w, tMin, tMax)`, whose inputs `x` and `w` are the ray origin o and direction w and whose outputs `tMin` and `tMax` are the entry and exit ray parameters of the intersection interval.

3.3.2 Fixed-Step Ray Marching

The fixed-step ray marcher is implemented identically for all three data structures. The ray is given in world space and is first transformed into the normalized object space $[-1, 1]^3$. Its intersection with the bounding box is computed by `ray_box_intersection`, which returns the entry and exit parameters $t_{\min}$ and $t_{\max}$. This interval is then validated. If $t_{\max} \leq 0$ the box lies entirely behind the ray origin, and if $t_{\min} > t_{\max}$ the ray misses the box. In both cases the marcher stops immediately and returns a transmittance of 1, meaning that no volume was encountered. Otherwise $t_{\min}$ is clamped to the ray origin, the object-space position is advanced to that entry point, and the marching function `transmittance_rm(x, w, d, step_size, scale)` is called.

Here `x` and `w` are the ray origin and direction in object space and `d` is the length of the interval, $t_{\max} - t_{\min}$. The parameter `step_size` is the length of one step along the ray and is set to 0,001. A smaller step size makes the result more accurate but requires more samples per ray. The parameter `scale` is a scalar factor applied to the accumulated optical depth before the transmittance is evaluated. It adjusts the opacity of the whole cloud without modifying the underlying data.

The starting position is offset by a uniform random fraction of one step. This reduces banding, although the effect is minor at a step size of 0,001. Each iteration evaluates the density σ at the current position through the sampler function and adds it to the accumulated optical depth after scaling by the step length, resulting in $\tau += \sigma \cdot \text{step_size}$. The sampler function is the only component that differs between the three data structures. The remaining distance `d` is then reduced by one step and the position is advanced by one step. If the scaled optical depth $\tau \cdot \text{scale}$ exceeds a fixed threshold of 20, the loop stops and returns a transmittance of 0. Once `d` is no longer positive the loop ends and returns $\exp(-\tau \cdot \text{scale})$. The threshold of 20 corresponds to a transmittance of $\exp(-20) \approx 2 \times 10^{-9}$, which is treated as a transmittance of 0.

3.3.3 Space-Skipping Ray Marching with DDA

This section describes a second implementation of the ray marcher. Rather than using a fixed step size throughout the entire volume, this version applies the DDA algorithm to skip regions that contain no data. Two variants are implemented. The two-level grid uses a standard 3D-DDA, whereas NanoVDB relies on a hierarchical DDA (HDDA). While the 3D-DDA only skips uniform regions of size B^3 , HDDA traverses the tree hierarchy and skips upper internal nodes, lower internal nodes, or single leaf nodes depending on occupancy.

Macro-Block DDA for the Two-Level Grid

A DDA on its own is only a cell traversal algorithm. Given a ray and a regular grid, it enumerates the grid cells the ray passes through in order. Applied to the macro blocks of the two-level grid and combined with the fixed-step marcher, it becomes a space-skipping algorithm. When the DDA enters an active block, the inner loop samples that block at the fixed step. When it enters an inactive block, it advances the ray parameter straight to the block boundary without evaluating any density. Figure 3.3 shows the effect in two dimensions. Both panels use the same sample spacing, so the samples taken with space-skipping are a subset of the samples of the plain marcher, with the empty region around the blob left out.

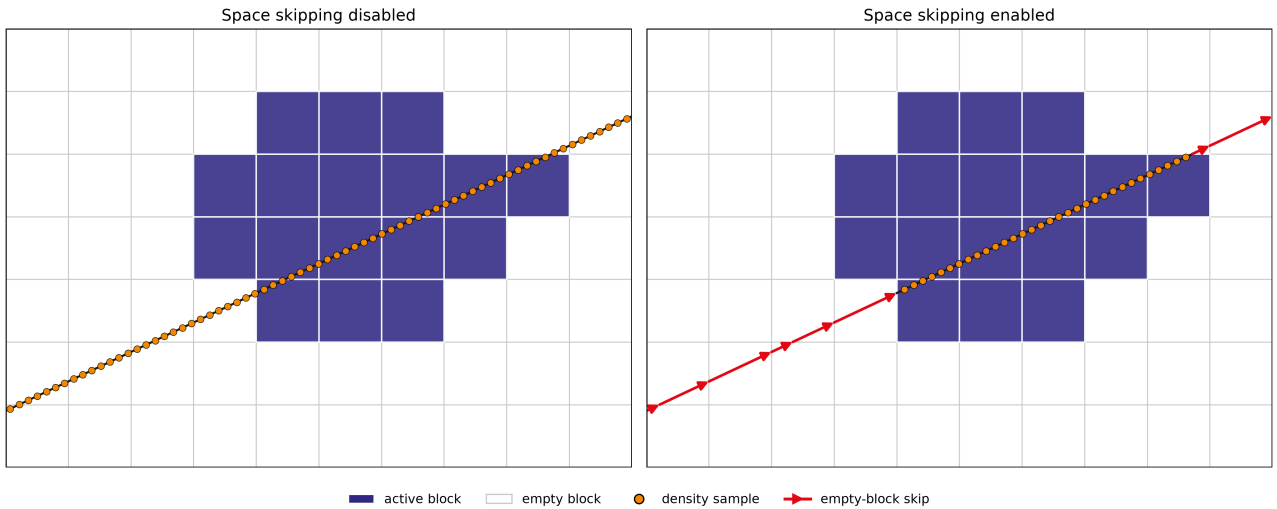


Figure 3.3 Space-skipping in two dimensions. On the left a fixed-step marcher and on the right a DDA space skipper.

The ray is transformed from object space into macro block index space so that one unit of distance along an axis corresponds to exactly one macro block, which makes the block containing a point the floor of that point and makes neighbouring blocks differ by one unit per axis. Whether a block is active is then a single fetch from the macro grid array.

Figure 3.4 shows one iteration. The ray is $r(t) = o + w t$ with origin o , direction w and parameter $t \geq 0$. The starting block c is the floor of o , here $(0, 0)$.

A few per-axis quantities are computed once before the loop. The vector `ray_inv` holds the component-wise inverse of the direction and is precomputed so the loop needs no divisions. Alongside this, `step` tracks the sign of each direction component as $+1$ or -1 , and `delta = |ray_inv|`, the amount of ray

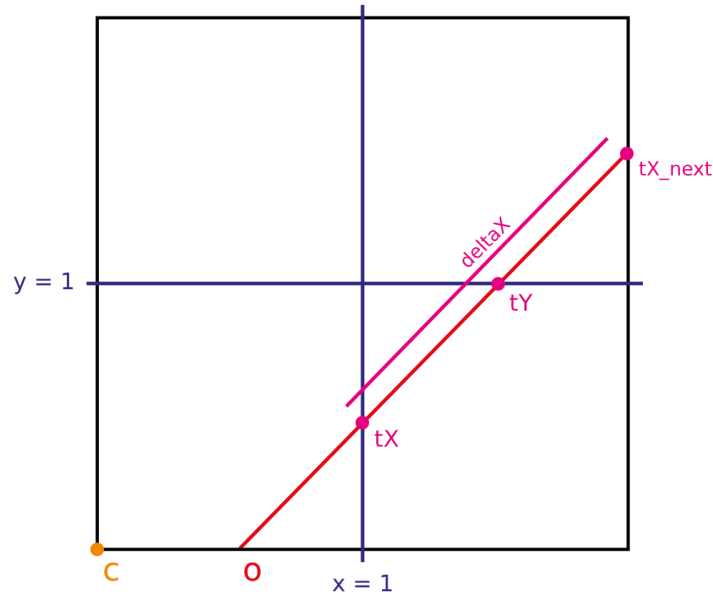


Figure 3.4 One step of the macro-block DDA. The blue lines $x = 1$ and $y = 1$ are the two candidate exits of block c . tX and tY are the ray parameters at which the ray meets those lines. Since tX is smaller, the ray leaves through $x = 1$, the block coordinate advances in x , and the next vertical crossing tX_{next} is found by adding the fixed spacing δx to tX .

parameter needed to cross one full block along an axis. Since the block spacing is normalized to 1, one block costs $\delta x = |1/w|$ per axis, shown as the cyan segment δx in the figure. The vector $\text{select} = 0.5 + 0.5 \cdot \text{step}$ is 1 for a positive component and 0 for a negative one, so $\text{planes} = c + \text{select}$ is the exit boundary of the current block on each axis, and $t_{next} = (\text{planes} - o) \cdot \text{ray_inv}$ is the ray parameter at which the ray meets each of those boundaries. In the figure both components of w are positive, so planes is $(1, 1)$ and t_{next} has components tX and tY .

Each iteration takes the smallest component of t_{next} , which is the boundary the ray reaches first. If the current block is active, the fixed-step marcher samples from the current position up to that parameter. If it is inactive, the ray parameter jumps straight to it. The block coordinate is then advanced by step on that one axis, and the same component of t_{next} is increased by its δx , while the other components stay unchanged because no boundary was crossed on those axes. In the illustrated step tX is smaller than tY , so the ray leaves through $x = 1$, c becomes $(1, 0)$, and tX is replaced by $tX + \delta x$, written tX_{next} . The loop repeats until the ray parameter reaches the end of the volume segment.

Since the block spacing is a constant 1, δx never changes, so each iteration costs one comparison to pick the axis and one addition to update it.

HDDA example in 2D

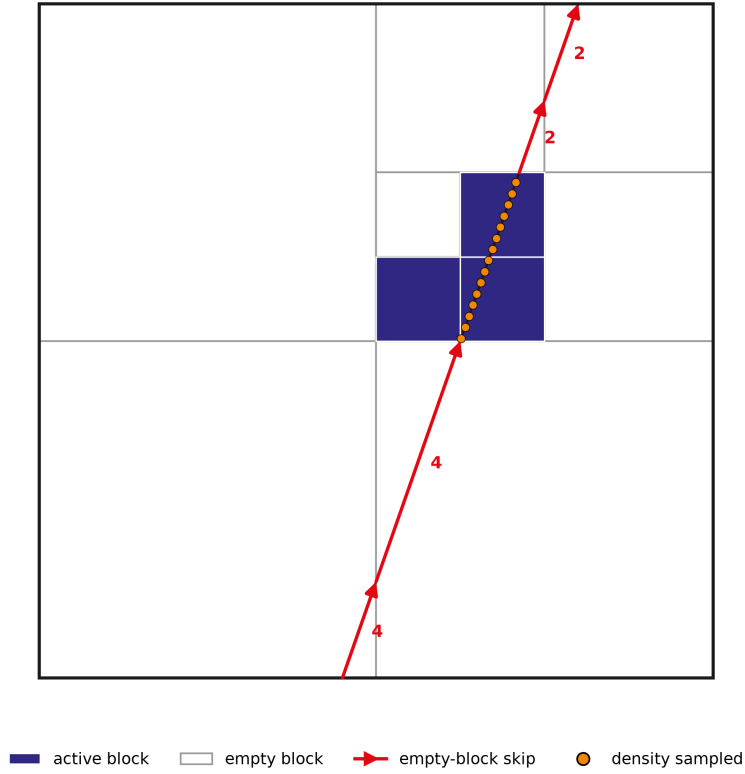


Figure 3.5 Hierarchical DDA in two dimensions.

Hierarchical DDA for NanoVDB

The two-level grid implementation in Section 3.3.3 operates on a uniform macro grid where each traversal step spans a single macro block. In contrast, NanoVDB organizes volumetric data into a hierarchical tree with multiple resolution levels (see Figure 3.5). Its hierarchical DDA variant, termed HDDA, traverses nodes whose dimensions vary depending on the local sparsity of the volume. In NanoVDB, a leaf node spans 8^3 voxels, a lower internal node spans 128^3 voxels, and an upper internal node spans 4096^3 voxels. Rays passing through extensive empty regions bypass large portions of the domain in very few traversal steps.

Rather than implementing the low level grid traversal algorithm manually, PNanoVDB offers its own HDDA implementation. It supplies the initialization, stepping, and state update functions, which internally track the direction signs, component step sizes, and next intersection parameters. The ray origin and direction are first converted into voxel index space as `idx_origin` and `idx_dir`. The initial voxel index `ijk` is computed by flooring the starting position. Calling `pnanovdb_readaccessor_get_dim` determines `dim`, which represents the side length of the largest empty node containing `ijk`, or evaluates to one when `ijk` resides within an active leaf. This dimension, the entry distance, and the ray exit limit `d` are then supplied to `pnanovdb_hdda_init`. In general it holds that $\text{dim} \in \{1, 8, 128, 4096\}$, where the voxels is within a leaf node if $\text{dim} = 1$.

In each iteration, the traversal evaluates `hdda.next`, which contains the parametric distances to the boundaries of the current node along all three coordinate axes. The minimum component, marks the exit

point `cell_exit`. If `hdda.dim` equals 1, the current region is an active leaf node and the inner loop samples density values using fixed step sizes, identical to the approach in Section 3.3.3. If the node is inactive, the ray parameter advances directly to `cell_exit` without sampling. The traversal state then updates via `pnanovdb_hdda_step`. Since the adjacent node may belong to a different level of the hierarchy, the new coordinate is queried with `pnanovdb_readaccessor_get_dim`, and `pnanovdb_hdda_update` adjusts the internal traversal parameters to match the new dimension. This update step represents the main distinction from the two-level grid DDA, where cell dimensions remain constant. The read accessor mitigates traversal overhead by caching tree levels, so queries in neighboring voxels avoid traversing down from the root node.

3.3.4 Sampling Coherence Scenarios

The cost of a single grid query is not constant. On the GPU, threads execute in warps that share L1 and L2 caches and memory-coalescing hardware, and the NanoVDB accessor additionally caches the last traversed node. A query therefore runs faster when the threads around it, and the queries that preceded it, fetched nearby voxels. To test this behavior, each implementation is measured with two additional sampling patterns. Each pattern is a tensor of coordinates that is generated ahead of time and handed to the sampling functions. The following paragraphs describe the two tensors.

Coherent linear traversal This tensor has shape $(256, 256, 256)$. Over an object space of $[-1, 1]^3$ it holds $256 \cdot 256 \cdot 256$ coordinates spaced evenly from -1 to 1 on each axis. The layout is row major with x as the fastest changing axis, so coordinates that are adjacent in memory are also adjacent in space. The tensor depends only on its shape, so it is identical across runs and every representation is queried with the same set of points.

Incoherent random access This tensor holds 256^3 coordinates drawn uniformly at random from $[-1, 1]^3$. The draw uses a fixed seed, so every representation is queried with the identical set of points and any measured difference corresponds to the data structure alone. Unlike the linear tensor, two coordinates that are adjacent in memory carry no spatial relation to one another.

Both tensors are consumed with one thread per coordinate. Thread i is mapped to row i of the tensor, so a warp of consecutive threads reads a contiguous block of rows. For the linear tensor that block is a run of voxels adjacent along x , whereas for the random tensor it is an unrelated set of positions. These two patterns can only be tested on the basic sampler implementations without DDA.

4 Results & Evaluation

4.1 Experimental Setup

The implementation chapter introduced seven traversal configurations. They combine four extinction samplers with two families of ray marchers. The four samplers are the dense grid, the two-level sparse grid, its ghost-voxel padded variant, and NanoVDB. Each sampler runs under the shared fixed-step marcher of section 3.3.2, which gives four fixed-step configurations. Three further configurations add space-skipping, the macro-block DDA of section 3.3.3 on the two-level grid and on its padded variant, and the hierarchical DDA of section 3.3.3 on NanoVDB. The dense grid carries no sparsity information and therefore has no space-skipping form. Table 4.1 lists all seven together with the names used throughout this chapter.

The four two-level configurations carry a block-size parameter and are measured at block sizes 2, 4, 8, 16 and 32. The dense grid, NanoVDB and NanoVDB-HDDA have no block-size parameter and are measured once. The marching step size is fixed at 0.001 for every configuration, and the camera position and cloud placement are held constant across all runs, so that only the data structure and the marcher change between measurements.

Every configuration is evaluated under a single 512×512 image rendered from one fixed camera. The four configurations that expose a three-dimensional point sampler, namely dense, two-level, two-level-padded and NanoVDB, are additionally evaluated under the linear and random point-query scenarios of section 3.3.4, each a batch of 256^3 coordinates.

Table 4.1 The seven traversal configurations evaluated in this chapter.

Name	Data structure	Marcher	Block size
dense	dense grid	fixed-step	none
two-level	two-level grid	fixed-step	2, 4, 8, 16, 32
two-level-padded	padded two-level grid	fixed-step	2, 4, 8, 16, 32
NanoVDB	NanoVDB	fixed-step	none
two-level-DDA	two-level grid	3D-DDA	2, 4, 8, 16, 32
two-level-padded-DDA	padded two-level grid	3D-DDA	2, 4, 8, 16, 32
NanoVDB-HDDA	NanoVDB	HDDA	none

The measurements use three clouds, which are all taken from the *cloudy* dataset [Leonard et al., 2025]. Every volume is trimmed on each spatial axis to the nearest lower multiple of 32 before use. The two-level grid requires each dimension to be divisible by its block size, and the same trimmed volume is shared across the whole block-size sweep, so it is trimmed to a multiple of the largest block size, 32, which tiles evenly for every block size from 2 to 32. The `.nvdb` export applies the identical trim, so all seven configurations operate on the same voxel data. Occupancy will be defined as the fraction of active voxels

within the trimmed volume. Table 4.2 lists the trimmed resolution, voxel count and occupancy of the three test clouds. Each is given a short label, A for the sparsest through to C for the densest, and the rest of this thesis refers to the clouds by these labels.

Table 4.2 The three test clouds, each cropped to a common $416 \times 192 \times 448$ box so that occupancy is the only variable. They are labeled A to C from sparsest to densest.

Label	Cloud	Resolution ($D \times H \times W$)	Voxels	Active voxels	Occupancy
A	ccloud_1107	$416 \times 192 \times 448$	35 782 656	7 709 836	21.5%
B	ccloud_1352	$416 \times 192 \times 448$	35 782 656	11 020 758	30.8%
C	ccloud_760	$416 \times 192 \times 448$	35 782 656	13 406 920	37.5%

Each cloud is stored as a .pt file and converted once into the dense, two-level, and NanoVDB representations. Figure 4.1 shows the three clouds side by side.

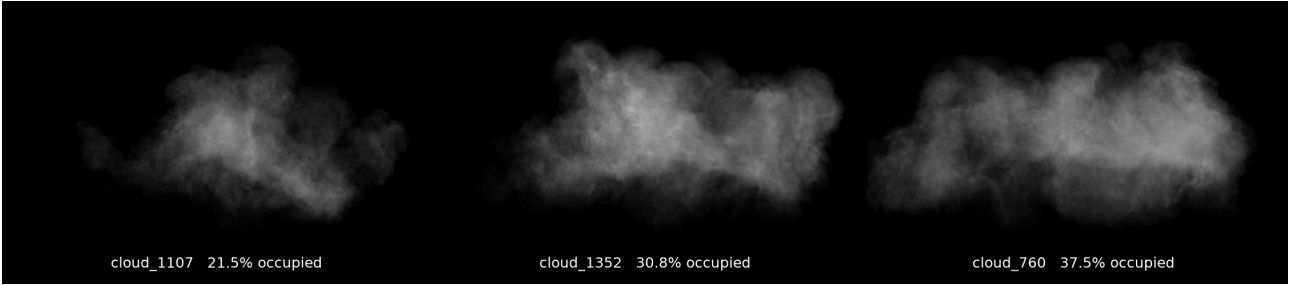


Figure 4.1 The three test clouds A, B and C.

All measurements run on an NVIDIA GeForce RTX 4070 Ti SUPER with 16 GB of memory, driver version 610.88, and on the operating system Windows 11. Compute shaders are written in GLSL 4.60 and compiled for Vulkan 1.3. The Python environment is Python 3.12 with PyTorch 2.5.1 on the CUDA 12.1 runtime and NumPy 2.4.6. Conversion of a volume to the .nvdb format uses NVIDIA's fVDB [Williams et al., 2024], and the resulting grids are loaded through NVIDIA Warp 1.16.0.

Each measurement performs 10 warm-up iterations, which absorb one-time pipeline build and allocation costs, followed by 50 timed iterations. One iteration is a full frame in the rendering scenario and one full batch of point queries in the point-query scenarios. The GPU is synchronized before and after each timed iteration and the elapsed wall-clock time is recorded in milliseconds. The mean and standard deviation reported for each configuration are computed across the 50 timed iterations.

4.2 Image Accuracy

Before rendering performance is measured, it must be established that the representations produce the same image. Since a stochastic jitter is applied to the start of every ray, an exact match is not attainable. The dense grid is therefore taken as the reference, and each of the six other representations is compared against it pixel by pixel. A pixel counts as matching when its transmittance differs from the reference by less than 10^{-3} , which absorbs most of the jitter noise. Figure 4.2 reports the result for cloud A. The fixed-step two-level grid and fixed-step NanoVDB reproduce the reference on every pixel. The other four

configurations, the fixed step padded grid and the three variants that skip empty space, each disagree on roughly 70 of the 262 144 pixels, which is below 0.03% of the frame.

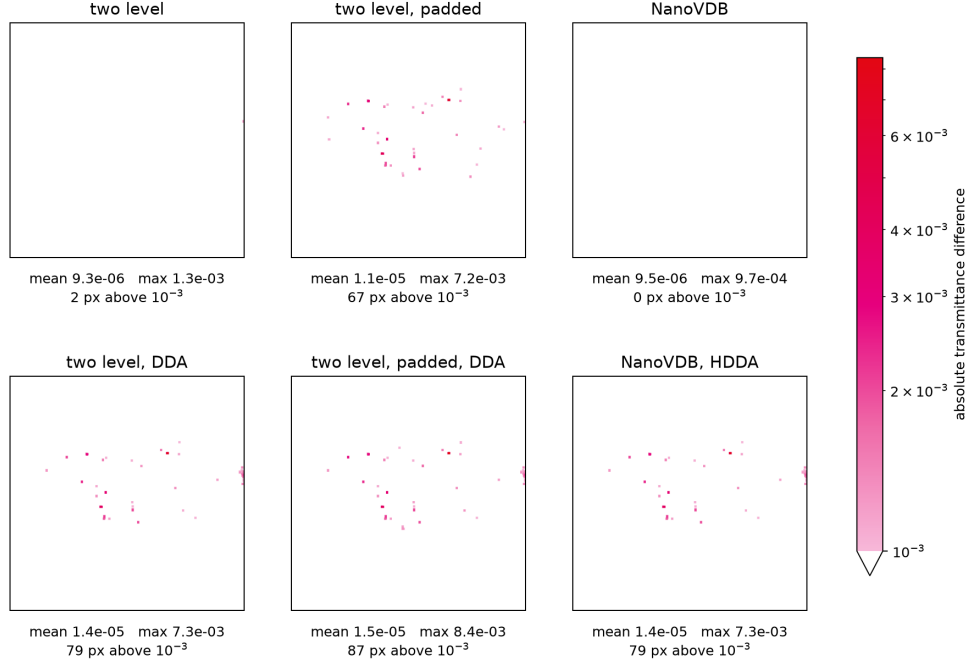


Figure 4.2 Per pixel absolute transmittance difference from the dense reference for the other six representations on cloud A, dilated for visibility and thresholded at 10^{-3} .

The number of differing pixels is small enough to ignore for the timing comparison, but it has a clear cause. Every differing pixel lies on the cloud silhouette, on a boundary between an active and an inactive block. A block is marked inactive only when all of its own voxels are zero, yet the reconstructed density is continuous across the grid. Inside the last half voxel of an inactive block the trilinear reconstruction is already above zero, because it interpolates toward the first voxel of the neighboring active block. The fixed step marcher samples this region and accumulates its contribution. A traversal that skips the inactive block advances the ray parameter straight to the block boundary and never evaluates it, so that thin slab of optical depth is lost. The same slab is lost again where a ray leaves an active block into an inactive one. The fixed-step padded grid shows the same behavior while not using space-skipping at all. It resolves all eight corners of a trilinear cell from the one block that contains the lower corner, so a cell whose lower corner lies in an inactive block reads the reserved zero block for every corner. Its reconstruction is then flat zero across the whole inactive block and never ramps toward the active neighbor, resulting in the same missing slab. Figure 4.3 illustrates the effect in one dimension.

Since the lost slab is at most half a voxel wide and the density there is a fraction of the neighboring value, the error stays well below the visible range and does not affect the performance results. It is a systematic bias of space skipping rather than random noise, so it would need to be corrected for an application that requires exact transmittance, for example by extending the fixed-step march one voxel into the skipped block at every active boundary.

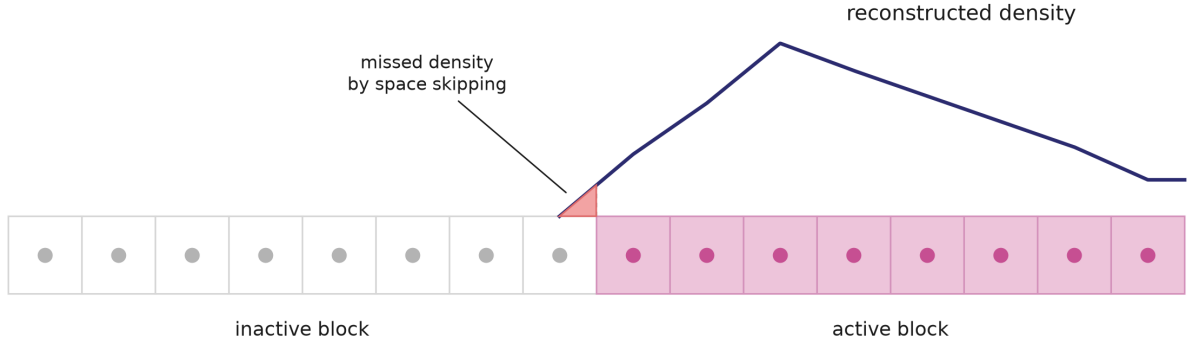


Figure 4.3 One-dimensional illustration of the border effect, showing the thin slab of optical depth lost when a space-skipping traversal jumps straight to a block boundary instead of sampling into it.

4.3 Memory Footprint

This section compares the size each representation occupies on the device, independent of any traversal. The dense grid stores one 32-bit floating-point per voxel, resulting in its size following directly from the resolutions in Table 4.2 rather than occupancy. The formula $D \cdot H \cdot W \cdot 4B$ yields a total size of 136.5 MiB for all clouds as they share the same dimensions. It is the baseline that every sparse representation is measured against.

4.3.1 NanoVDB

NanoVDB stores only the leaves of its tree that contain data, together with the internal nodes above them, and needs no parameter to be chosen. For the three clouds its size is 50.5 MiB, 65.2 MiB and 76.2 MiB. Table 4.3 lists these figures next to the dense sizes and the compression they represent.

Table 4.3 Dense grid and NanoVDB size per cloud, with the compression of NanoVDB relative to the dense grid.

Cloud	Occupancy	Dense	NanoVDB	Compression
A	21.5%	136.5 MiB	50.5 MiB	63.02%
B	30.8%	136.5 MiB	65.2 MiB	52.25%
C	37.5%	136.5 MiB	76.2 MiB	44.17%

The compression shrinks as occupancy rises. NanoVDB reduces the sparsest cloud A by 63.02% and the densest, C, by only 44.17%, with a mean of 53.15% over the three. This follows from how NanoVDB spends storage. It divides the volume into 8^3 blocks and keeps one leaf node for every block that holds at least one active voxel, and nothing for a block that is completely empty. Each stored leaf is the same fixed size, whether one of its 512 voxels is active or all of them. So the buffer size is set by how many 8^3 blocks contain any part of the cloud, not by the total number of active voxels. A denser cloud spreads into more of these blocks, so more leaves are stored and the buffer grows. If the cloud filled the whole box, every block would hold a leaf and NanoVDB would be a little larger than the dense grid, since it stores all voxels and additionally a tree overhead.

4.3.2 Two-Level Grid

Unlike the dense grid and NanoVDB, the two-level grid has a parameter, the block size, and its size depends strongly on it. Figure 4.4 plots the total size of the two-level grid against the block size for each cloud. It falls as the block size grows from 2 to 4 and rises again from 4 to 32, so every cloud reaches its minimum at block size 4, at 42.7 MiB, 56.3 MiB and 66.1 MiB. Notably, at block size 8 the two-level grid nearly matches NanoVDB in size across all clouds, since both representations share an identical leaf granularity of 8^3 voxels. The unpadded two-level grid remains slightly smaller here because its flat macro array imposes less structural overhead than NanoVDB's multi-level tree hierarchy.

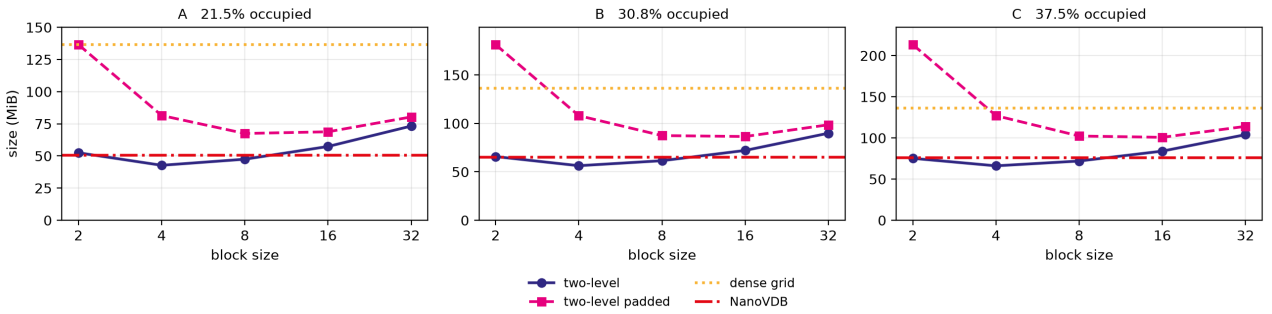


Figure 4.4 Total size of the two-level and padded two-level grids against block size, with the dense grid and NanoVDB as horizontal references. One panel per cloud, ordered from sparse to dense.

Figure 4.5 splits the two-level size into its two arrays and shows why it behaves this way. The macro array stores one 32-bit integer per block, so its size is proportional to the block count. With B the block size and $D \times H \times W$ the trimmed resolution, the block count is

$$N = \frac{D \cdot H \cdot W}{B^3},$$

and doubling the block size divides it by eight,

$$\frac{D \cdot H \cdot W}{(2 \cdot B)^3} = \frac{D \cdot H \cdot W}{8 \cdot B^3} = \frac{N}{8}.$$

The **macro** array therefore falls from 17.06 MiB at block size 2 to below 0.3 MiB at block size 8 and is negligible beyond that. For cloud B the **micro** array rises from 48.67 MiB at block size 2 to 89.88 MiB at block size 32. To explain this, the block fill fraction is introduced. It is the number of active voxels divided by the number of voxels stored across all active blocks, which is the average share of a stored block that holds data. Table 4.4 lists it for cloud B, together with the number of active blocks and the micro array size. Since cloud B contains a fixed number of 11 020 758 active voxels, the micro array must store at least this many voxels. The two-level grid aims to waste as little memory as possible on empty voxels inside active blocks, which is measured by the fill fraction. Larger blocks reduce the total number of active blocks, but each block covers more volume and contains a higher share of empty space. As a result, the fill fraction drops from 86% at block size 2 to 47% at block size 32. On average, the number of empty voxels stored per block rises from about one to around 17 000, which makes the micro array grow. The total size is the

sum of a term that falls steeply and a term that rises, so it is smallest where the macro array has already become negligible but the micro array has not yet grown much, at block size 4.

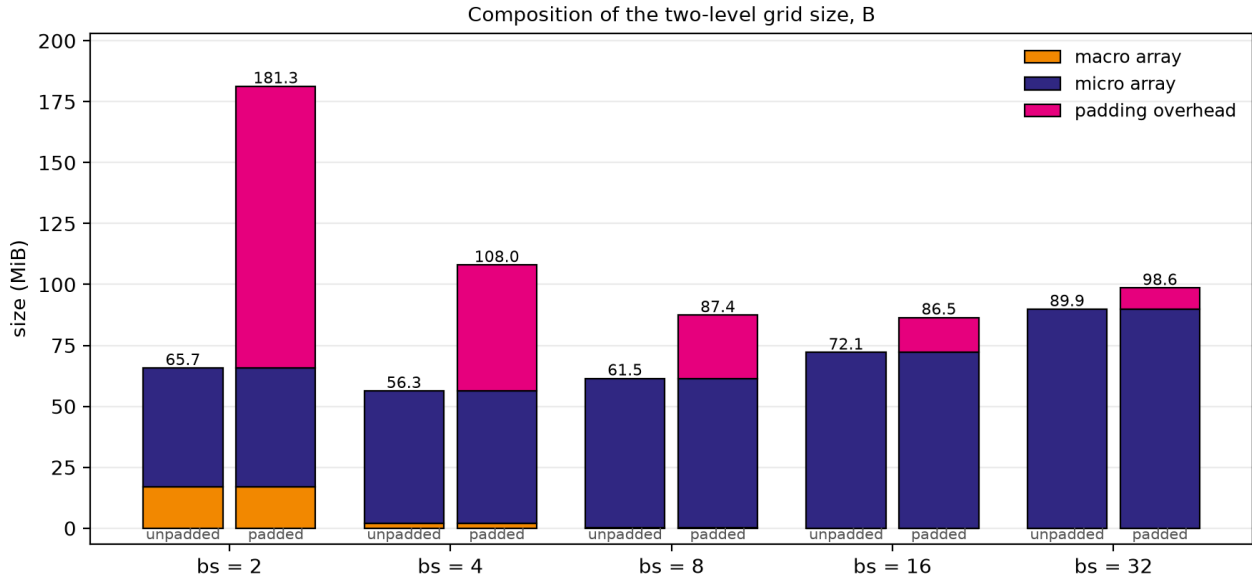


Figure 4.5 Macro array, micro array and padding overhead of the two-level grid for cloud B.

Table 4.4 Block fill fraction for cloud B. For each block size, the number of active blocks, the average share of a stored block that holds data, and the resulting micro array size.

Block size	Active blocks	Fill fraction	Micro array
2	1 594 916	86%	48.7 MiB
4	221 988	78%	54.2 MiB
8	31 341	69%	61.2 MiB
16	4 611	58%	72.1 MiB
32	718	47%	89.9 MiB

The padded variant stores a one voxel border on the positive side of each block, so a block holds $(B + 1)^3$ voxels instead of B^3 . The macro array is unchanged, and the micro array is larger by the factor $\left(\frac{B+1}{B}\right)^3$. Table 3.1 lists this factor and the relative memory overhead it produces for each block size. The overhead is around 238% at block size 2 and falls to around 10% at block size 32, so the border is expensive for small blocks and almost free for large ones. It appears as the gap between the two curves in Figure 4.4, which narrows toward the right, and as the top segment of each padded bar in Figure 4.5. Since padding is only affordable at large block sizes, the padded grid reaches its minimum at a larger block size than the unpadded one, at block size 8 for A and block size 16 for B and C, with 67.5 MiB, 86.5 MiB and 100.7 MiB.

Table 4.5 compares each grid at its optimal block size with the dense grid and NanoVDB. At block size 4 the unpadded two-level grid is 69%, 59% and 52% smaller than the dense grid for the three clouds, which shows the same behavior as NanoVDB, a compression rate that falls as the cloud gets denser. Despite this trend the unpadded grid still beats NanoVDB for every cloud, by 15%, 14% and 13%. The padded version adds an overhead of 58%, 54% and 52% on top of the unpadded size, which pushes it to 34%, 33% and 32% larger than NanoVDB, roughly a third for every cloud, while still being smaller than the

dense representation for all clouds. These numbers result in an average compression rate of 60% for the unpadded and 38% for the padded version, while NanoVDB showed an average compression rate of 53%.

Table 4.5 Smallest two-level size per cloud with the block size that achieves it, next to the NanoVDB, padded two-level and dense sizes. The representations are listed in ascending order of size, which is the same order for every cloud.

Cloud	Two-level	NanoVDB	Two-level-Padded	Dense
A	42.7 MiB (bs 4)	50.5 MiB	67.5 MiB (bs 8)	136.5 MiB
B	56.3 MiB (bs 4)	65.2 MiB	86.5 MiB (bs 16)	136.5 MiB
C	66.1 MiB (bs 4)	76.2 MiB	100.7 MiB (bs 16)	136.5 MiB

4.4 Point-Query Throughput

4.4.1 Linear Query

The linear query measures the four representations with a three-dimensional point sampler under the most coherent access pattern. The input is a uniform grid of 256^3 coordinates covering the object space $[-1, 1]^3$. The x axis varies fastest, so coordinates that are neighbors along x are stored next to each other in the input tensor. Figure 4.6 shows the time to return the density at all 256^3 coordinates. As block size did not show any variation in time, only block size 8 will be used as comparison. The dense grid, the two-level grid and the padded two-level grid all take about 0.79 to 1.02 ms for the three clouds. NanoVDB is two to three times slower, ranging from 1.96 ms on cloud A to 2.50 ms on cloud C. NanoVDB clearly shows a relationship to occupancy while the other three representations are almost constant across all clouds. A fifth representation was measured with this linear query named *null sampler*. This sampler takes in a coordinate and does not perform any memory fetch or indirection. It shows the baseline and therefore tracks any overhead that is not caused by memory fetches or arithmetic operations.

Each thread evaluates one coordinate, so the query dispatches 256^3 threads, grouped into warps of 32. Coordinates are assigned to threads in order, so the 32 threads of a warp cover 32 positions that are adjacent along x , and a full row of 256 positions is covered by 8 warps. Every representation fetches the eight voxels of the enclosing cell for the trilinear interpolation. Since the 32 sample positions of a warp are adjacent, these fetches overlap and fall within a small region of memory, so the whole warp reads only a few cache lines. This behavior can be seen by the very small slowdown the dense, two-level and padded two-level variants have against the null sampler variant.

Since NanoVDB is the only representation whose cost grows with occupancy, the variable that changes across the three clouds has to be identified before this behavior can be explained. The dense grid reads the same array for every cloud, so its cost is constant. For NanoVDB a sample that falls in empty space terminates early at an upper or lower tile, whereas a sample inside an occupied region descends through all four levels to a leaf. As occupancy rises the fraction of samples that reach a leaf rises with it, from roughly 35% for cloud A to 53% for cloud C. This fraction is the only thing that changes, so the slowdown must originate in whatever an active sample does that an inactive sample does not. One difference is the descent. A trilinear sample reads eight corner values, and the NanoVDB read accessor caches the

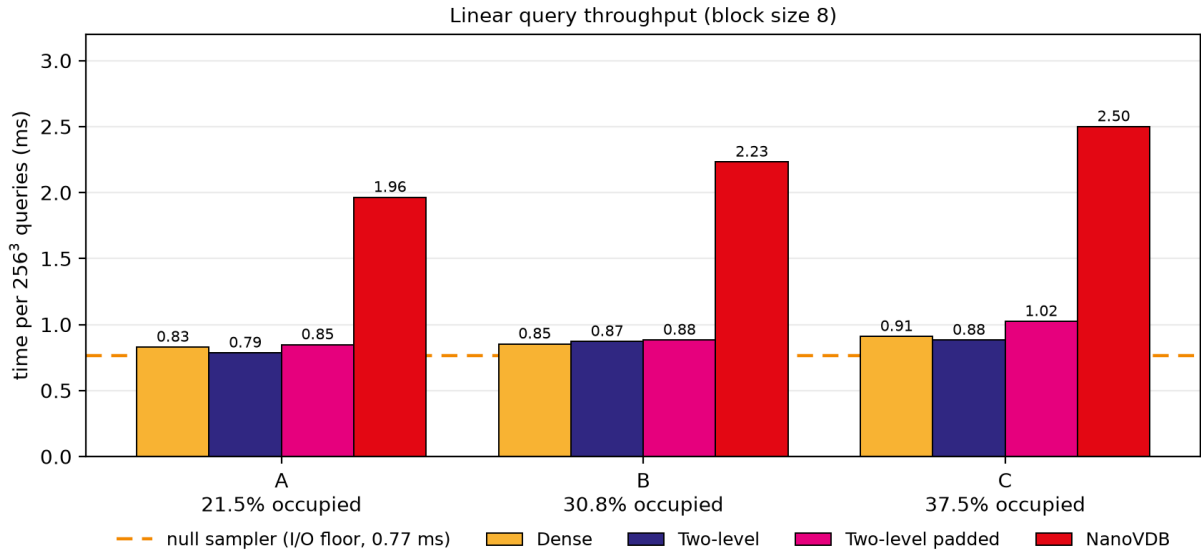


Figure 4.6 Time for one batch of 256^3 linear point queries, per cloud and per representation, including the null sampler baseline.

path to the last visited node, so when the eight corners share a leaf they are reached by one four level descent rather than eight. If descending to the last level was the source of the cost, that single cached descent would already have to be expensive enough to appear in the measurements. The other difference is the eight reads themselves. When a sample lies in an inactive region the eight corner lookups resolve to the same tile address and collapse to one memory access, whereas inside a leaf they address eight separate entries of the leaf array. This places the cost in the per corner memory traffic, and it happens only for active samples. To separate the two, a reduced NanoVDB sampler, called `OneFetchNoTrilinear`, was implemented that performs the full per sample descent but issues a single memory read, at the lower corner of the enclosing cell, instead of the eight corners, with no interpolation. Under the linear query this sampler matches the dense baseline within about 0.05 ms above the query floor, so the single descent contributes no measurable cost. The gap between `OneFetchNoTrilinear` and the full sampler is therefore assignable to the seven additional corner reads. The slowdown with occupancy is therefore a property of the per corner memory traffic inside a leaf and not of tree traversal.

4.4.2 Random Query

The other extreme is fully incoherent access. The input tensor is again of size 256^3 , this time filled with points drawn uniformly at random from $[-1, 1]^3$ with a fixed seed, so every representation receives the same points. Neighboring threads in a warp now sample unrelated parts of the volume, so the eight corner reads of one thread almost never share a cache line with those of its neighbor. Figure 4.7 shows the result.

Every representation is now between roughly two and twelve times slower than the null floor of about 0.6 ms. Streaming the coordinates and densities is no longer the limit. The cost is now the grid fetches themselves, and most of them miss the cache. The dense grid achieves a constant time of 6.50 ms for every cloud. NanoVDB once again shows a pattern that matches rising occupancy with 6.05 ms, 7.59 ms

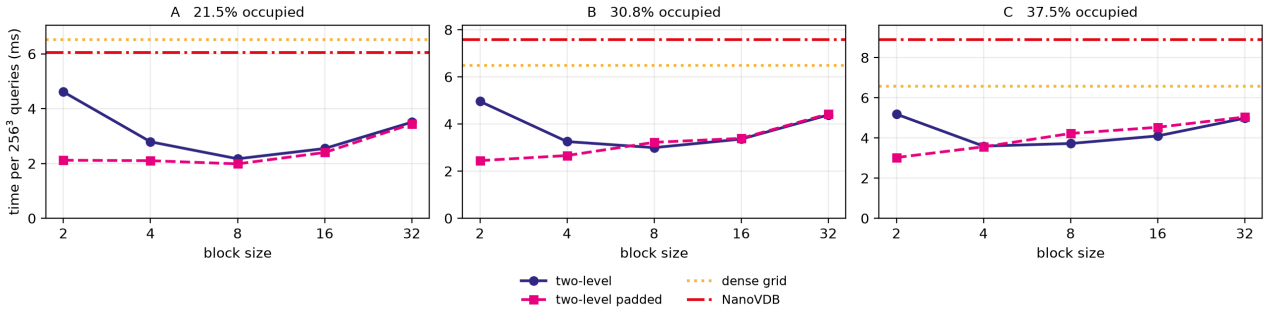


Figure 4.7 Time for one batch of 256^3 random point queries against block size, for the two-level and padded two-level grids, with the dense grid and NanoVDB as horizontal references. One panel per cloud, ordered from sparse to dense. Block size 1 is omitted.

and 8.91 ms. For cloud A this results in NanoVDB being quicker than the dense grid while for clouds B and C NanoVDB is slower than the dense grid. The dense grid once again does not adapt to the opacity of the data and therefore always fetches memory which explains the constant time. The dense grid is around 8 times slower under the random query compared to the linear one. NanoVDB on the other hand is only 3.5 times slower for the random query. This further shows that NanoVDB already has memory bound issues under the linear query.

Section 4.4.1 showed directly that this is a fetch problem rather than a descent problem, since the simplified sampler `OneFetchNoTrilinear`, which keeps the full descent but reads only one corner, already matches the dense grid under the linear query. The same test was repeated here under random access, together with a second sampler, `OneFetch`, which restores the full trilinear interpolation on top of that same single fetch in order to isolate the cost of the interpolation arithmetic from the cost of the seven extra reads. Table 4.6 lists both against NanoVDB and the dense grid. Averaged over the three clouds, `OneFetchNoTrilinear` is 3.91 times faster than the full sampler and `OneFetch` is 3.75 times faster. This confirms the eight leaf reads as the cost under random access as well, and the small gap between the two variants shows that the interpolation arithmetic itself is close to free.

Table 4.6 Dense and the three NanoVDB samplers under random query. Dense and NanoVDB are the values from Figure 4.7.

Cloud	OneFetch	OneFetchNoTrilinear	NanoVDB	Dense
A	1.62 ms	1.51 ms	6.05 ms	6.53 ms
B	2.08 ms	2.00 ms	7.59 ms	6.48 ms
C	2.36 ms	2.25 ms	8.91 ms	6.58 ms

The two-level grid is faster than both references at every block size. Its unpadded cost first falls and then rises with block size, lowest around four to eight. The macro array holds one entry per block, so a small block size makes it large enough that the lookup itself starts to miss. A large block size shrinks the macro array to almost nothing but inflates the blocks, which then have a low fill fraction, so the block pool grows back towards the size of the dense array and a sample's eight corners are spread across a 32^3 block, which limits memory locality. Given these two effects, the best unpadded runs are 2.17 ms, 3.00 ms and 3.59 ms at block sizes four to eight for the three clouds. Padding helps most where the blocks are small.

At a block size of two the trilinear interpolated cell is as wide as a whole block, so without padding most samples cross a block boundary and the unpadded sampler resolves several corners through separate macro and micro fetches, while the ghost layer keeps all eight voxels in a single block. For example this takes cloud A at a block size of two from 4.61 ms down to 2.12 ms. The effect fades as the block size grows and is largely gone by a block size of eight. The best configuration under random access is the padded two-level grid at a block size between two and eight, at about 2 ms to 3 ms, which is three to two times faster than the dense grid. Table 4.7 lists for all cloud the best configurations of each representation with the exact timings.

Table 4.7 Fastest configuration per cloud under the random query for each representation, with the winning block size shown for the two-level variants.

Cloud	Two-Level-Padded	Two-Level	Dense	NanoVDB
A	1.99 ms (bs 8)	2.17 ms (bs 8)	6.53 ms	6.05 ms
B	2.45 ms (bs 2)	3.00 ms (bs 8)	6.48 ms	7.59 ms
C	3.02 ms (bs 2)	3.59 ms (bs 4)	6.58 ms	8.91 ms

4.5 Rendering Performance

4.5.1 Fixed-Step Marching

The next section examines rendering performance under fixed-step ray marching. The measurements are done on a single 512 by 512 frame and reported as the mean over all iterations. Figure 4.8 shows the results with a fixed block size of 8 as it made no difference in the measurements. Before looking at the specific clouds there is a trend that repeats within every cloud. The dense grid and the padded two-level grid perform best and stay within about ten percent of each other, at 1.60 ms and 1.47 ms on cloud A, 1.58 ms and 1.55 ms on cloud B, and 1.60 ms and 1.57 ms on cloud C. The unpadded two-level grid comes next at close to twice that, with 2.84 ms, 2.88 ms and 2.90 ms. NanoVDB is the slowest on every cloud at 14.29 ms, 16.44 ms and 18.66 ms, which is nine to twelve times the dense grids performance.

The dense grid once again builds a baseline performance as there is no special treatment for inactive voxels and it performs the same interpolation at each step which results in an almost constant time across all three clouds. Excluding NanoVDB, the unpadded two-level grid is the slowest representation at just under 2.90 ms for all clouds. The padded version achieves the best performance across all clouds with around 1.53 ms, which is around a two times speedup against the unpadded version. This shows that the slowdown comes from the sample positions where the eight corners fall in different blocks, which sit far apart in memory and do not coalesce into a single read. The padded grid stores all eight corners in one block, and the near two times speedup follows from that change alone. NanoVDB shows not only the slowest performance but also indicates a relationship with occupancy. With 14.29 ms to 18.66 ms it is around nine to twelve times slower than the dense grid. The NanoVDB performance can once again be explained with the pure sample cost it takes to fetch the eight voxels. In order to understand the magnitude of difference between the coherent linear sampling and the fixed-step ray sampling Table 4.8 shows the slowdown of NanoVDB against the dense grid aligned with the null sampler base cost. The adjusted

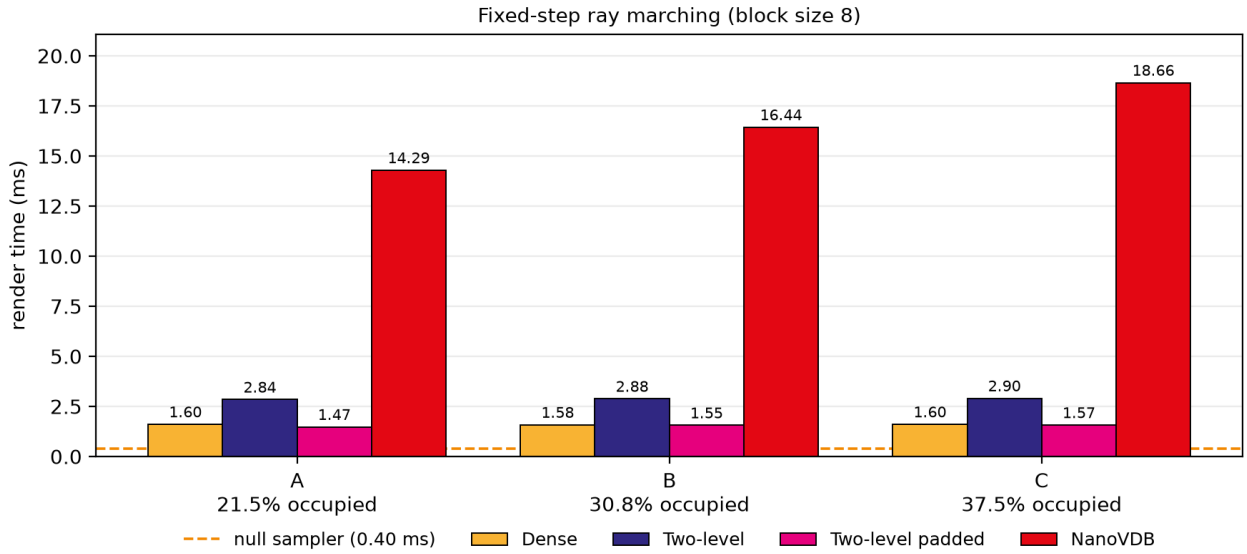


Figure 4.8 Fixed-step rendering time per cloud and representation at block size 8.

slowdown is about the same for the coherent linear query and the ray march, roughly fourteen to sixteen times. The dense grid turns the coherence of the linear sweep into almost no sampling cost, 0.09 ms above the floor, while NanoVDB still spends 1.45 ms, so NanoVDB gains almost nothing from access locality.

Table 4.8 Dense and NanoVDB on cloud B with the null sampler base cost removed.

Query	Null floor	Dense		NanoVDB		NanoVDB / Dense
		Raw	Adjusted	Raw	Adjusted	
Linear	0.77 ms	0.86 ms	0.09 ms	2.23 ms	1.46 ms	16.2×
Ray March	0.40 ms	1.58 ms	1.18 ms	16.44 ms	16.04 ms	13.6×

4.5.2 Space-Skipping Impact

This section examines the impact of DDA space-skipping algorithms on the render performance. For the two-level implementations a 3D-DDA version is used and for NanoVDB a HDDA version is used which can skip at different step sizes. Figure 4.9 illustrates the measurements against their respective fixed-step performance. The dense grid is still included as a baseline and did not use any new optimization. Space-skipping helps every representation, but by how much depends entirely on how much empty space there is to skip, so the effect shrinks as the clouds increase in occupancy. Table 4.9 lists the reduction against each representation's fixed-step time. The two-level grid gains the largest performance increase on cloud A, where it saves 35.2%, but only gains a speed up of 13.2% by cloud C. The padded grid shows the same pattern more sharply, from 28.1% down to only 4.9%. NanoVDB falls from 21.7% to about 10%. This confirms that space-skipping is not a fixed multiplier on render time. It is a saving on the empty fraction of the volume specifically, and it is largest exactly where fixed-step marching wastes the most work stepping through inactive voxels.

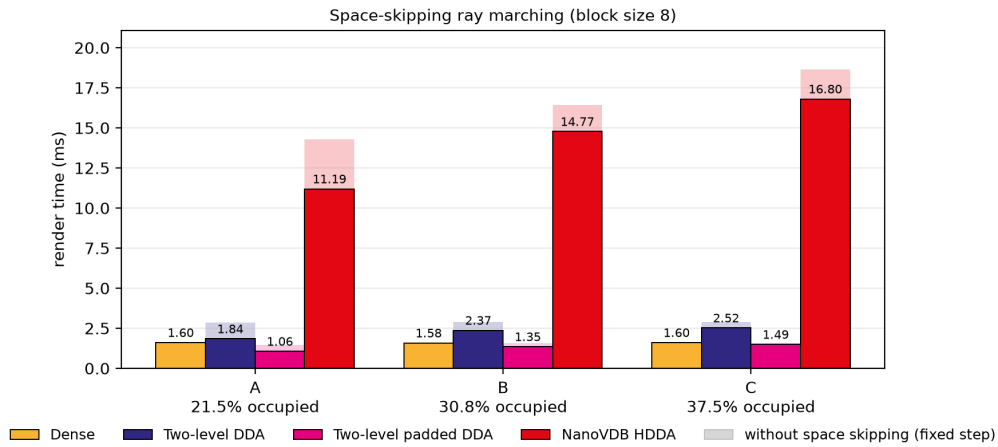


Figure 4.9 Space-skipping render time at block size 8 for the two-level grid with 3D-DDA, the padded two-level grid with 3D-DDA, and NanoVDB with HDDA.

Table 4.9 Reduction in render time from adding space-skipping to each representation's fixed-step marcher.

Cloud	Two-level	Two-Level-Padded	NanoVDB
A	35.2%	28.1%	21.7%
B	17.7%	13.2%	10.1%
C	13.2%	4.9%	10.0%

Figure 4.10 looks at the two 3D-DDA representations across block size. NanoVDB has no block size and is left out of this comparison. Both curves are lowest around block size four to eight and rise again on either side, the same tradeoff already seen for the point queries. The unpadded grid stays above the dense grid at every block size on all three clouds. The padded grid stays below the dense grid from block size two to sixteen on all three clouds, and at block size 32 it is still below dense on cloud A, ties it on cloud B, and above it on cloud C.

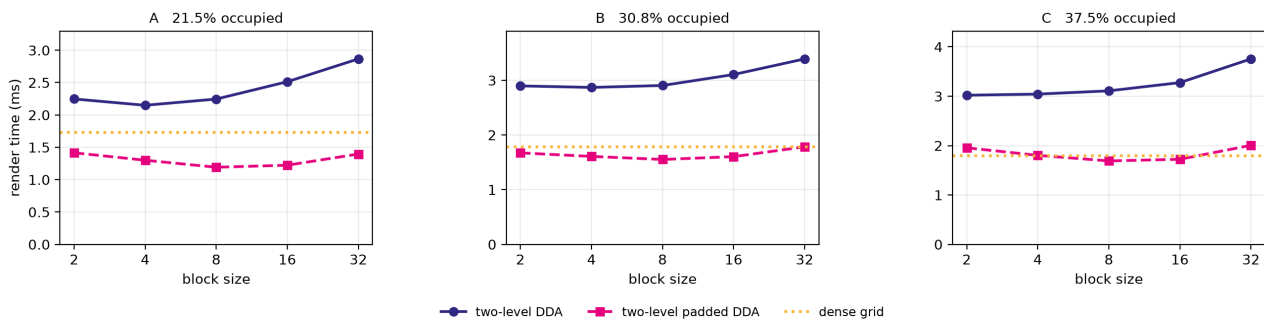


Figure 4.10 Space-skipping render time against block size for the two-level and padded two-level grids, one panel per cloud, with the dense grid as a single horizontal reference.

4.6 Summary

Table 4.10 collects the best performing representation in each discipline evaluated in this chapter, for all three clouds, together with its percentage change against the dense grid. The linear query is left out since it is effectively a tie, dense, two-level and padded all sit within about 0.1 ms of one another on every cloud, well inside measurement noise, and only NanoVDB stands clearly apart as the slowest. No single configuration wins everywhere among the rest. The unpadded two-level grid is the most compact representation on every cloud, roughly half the size of dense, while the padded two-level grid is the fastest representation in every remaining timing discipline on every cloud, though its margin over dense narrows down on the two rendering disciplines as the clouds increase in occupancy. The choice between the two two-level variants is therefore a choice between memory and speed rather than a clear winner. The unpadded grid is 36 to 37% smaller than the padded one at their respective best block sizes, which matters when the volumetric data size itself is the binding constraint. For every performance discipline in this chapter, the padded grid is faster than the dense grid.

Table 4.10 The fastest, or for memory the smallest, representation per discipline and cloud.

Discipline	Cloud A	Cloud B	Cloud C
Memory	Two-level (42.7 MiB, −69%)	Two-level (56.3 MiB, −59%)	Two-level (66.1 MiB, −52%)
Random	padded (1.99 ms, −70%)	padded (2.45 ms, −62%)	padded (3.02 ms, −54%)
Ray March	padded (1.47 ms, −8%)	padded (1.55 ms, −2%)	padded (1.57 ms, −2%)
DDA	padded (1.06 ms, −34%)	padded (1.35 ms, −15%)	padded (1.49 ms, −7%)

To further summarize the impact of the ghost-voxel layer for the two-level grid, table 4.11 illustrates the difference in memory and performance for both implementations. If a high performance in rendering is the goal and space-skipping is enabled, this results in a performance gain of 43% while an additional memory overhead of 54%.

Table 4.11 Unpadded versus padded two-level grid on cloud B, each at its own best block size for memory and at block size 8 for every timing discipline.

Metric	Two-level	Two-Level-Padded	Change
Memory footprint	56.3 MiB (bs 4)	86.5 MiB (bs 16)	+54%
Linear query	0.87 ms	0.88 ms	+1%
Random query	3.00 ms (bs 8)	2.45 ms (bs 2)	−18%
Fixed-step render	2.88 ms	1.55 ms	−46%
Space-skipping render	2.37 ms	1.35 ms	−43%

NanoVDB’s memory footprint is a middle ground between the unpadded and padded two-level implementations. While it compresses the three clouds for about 50%, its still beaten by the two-level representation. In performance, NanoVDB only managed to beat the dense representation in random access for cloud A. In all other measurements NanoVDB clearly scored the last place. In space-skipping it also showed less performance boost compared to the two-level representation.

5 Conclusion, Limitations and Future Work

5.1 Conclusion

This thesis investigated whether a two-level sparse grid can combine the space-skipping of hierarchical structures like NanoVDB with the low per-sample lookup cost of a dense grid. Specifically, it evaluated whether this simpler layout matches or exceeds NanoVDB in both memory footprint and query performance. Furthermore, it examined whether the overhead of a deeper tree hierarchy justifies itself in practice when restricted to bounded, single-frame clouds.

Comparing four volumetric representations — a dense grid, a two-level sparse grid, its ghost-voxel padded variant, and NanoVDB — demonstrated that no single structure dominates across all metrics. Across three clouds spanning 21.5% to 37.5% occupancy, memory footprint, point-query throughput, and rendering performance varied substantially. The unpadded two-level grid proved to be the most compact representation overall, requiring roughly half the memory of the dense grid. Meanwhile, the padded two-level grid was the fastest representation across all timing benchmarks (Table 4.10). Consequently, balancing memory footprint against rendering speed represents a direct trade-off between both two-level variants rather than yielding a single universal winner. In either form, the two-level grid provided competitive or superior compression relative to NanoVDB alongside throughput that the hierarchical tree structure could not match. These measurements answer the first research question, demonstrating that a two-level sparse grid is a well-suited representation for this specific setting.

In contrast, NanoVDB was the slowest representation in every timing benchmark. Memory footprint was the only metric where it did not place last. It achieved a better compression than the padded two-level grid while falling behind against the unpadded version. Under this implementation and sampling scenario, PNanoVDB (the GLSL port used throughout this work) offered neither a memory nor a performance advantage over the simpler two-level sparse grid. This resolves the second research question, indicating that the overhead of a general-purpose hierarchical tree provides no tangible benefits for static, bounded cloud volumes. However, these findings must be interpreted within the specific constraints of the test setup, as detailed in the next section.

5.2 Limitations

Several assumptions and implementation choices limit the extent to which the results of this thesis can be generalized. The three test clouds spanned from 21.5% to 37.5% occupancy. This range is sufficient to demonstrate occupancy-dependent effects, but not extreme cases. Museth [2013] used a narrow-band level set to showcase OpenVDB and its occupancy is usually far below 20%. NanoVDB already

outperformed the dense grid under random access on the sparsest cloud tested here (Section 4.4.2), so it would likely perform even better with a genuinely sparse dataset. Another limitation lies in the sampler implementation. PNanoVDB ships with no interpolating sampler of its own. The most straightforward way to build one is with eight independent leaf fetches. This thesis implemented this method, and the measurements in Section 4.4.1 identified it as the dominant cost. The original NanoVDB library offers a sampler, `SampleFromVoxels`, that caches the eight corner values of the last visited cell. Consequently, consecutive samples inside the same cell are free. This optimization is not provided by PNanoVDB, nor was it re-implemented in this thesis. The size of the bounding box is a third limitation. NanoVDB’s highest level, the root, is an array whose entries each cover a 4096^3 region. All three test clouds fit easily inside a single region of this size, so NanoVDB’s structure always held one fixed root entry rather than utilizing many root nodes. Larger and sparser volumes were not evaluated in this thesis. Therefore, any performance cost associated with this setting remains untested.

5.3 Future Work

The limitations above each point directly to an extension. Repeating this comparison on volumes with occupancy in the low single digits, closer to the regime that NanoVDB was designed for, would reveal whether the observed ranking reverses once the dense grid is no longer a viable baseline. Implementing a caching sampler for PNanoVDB, mirroring the native `SampleFromVoxels` behavior, would separate how much of NanoVDB’s measured cost is inherent to the format and how much is specific to this thesis’s sampler implementation. Finally, testing a scene whose bounding box exceeds a single 4096^3 root region would directly exercise NanoVDB’s root level and test whether it provides an advantage for larger scenes. Beyond these three, actual GPU profiling with a tool like *NSight* [NVIDIA Corporation, 2026b] from NVIDIA would give real information on cache hit rate and memory overhead, information this thesis could not obtain because the framework used here did not support a connection to *NSight* out of the box. More access patterns could also be tested, such as ray tracing with secondary rays.

Bibliography

- John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. *Proceedings of Euro-Graphics*, 87, 08 1987.
- Johanna Beyer, Markus Hadwiger, and Hanspeter Pfister. State-of-the-art in gpu-based large-scale volume visualization. *Computer Graphics Forum*, 34(8):13–37, 2015. doi: <https://doi.org/10.1111/cgf.12605>. URL <https://onlinelibrary.wiley.com/doi/abs/10.1111/cgf.12605>.
- Ian Buck. Implementing efficient parallel data structures on GPUs. In Matt Pharr and Randima Fernando, editors, *GPU Gems 2: Programming Techniques for High-Performance Graphics and General-Purpose Computation*, chapter 33, pages 521–534. Addison-Wesley Professional, 2005. URL <https://developer.nvidia.com/gpugems/gpugems2/part-iv-general-purpose-computation-gpus-primer/chapter-33-implementing-efficient>.
- Douglas Comer. Ubiquitous b-tree. *ACM Comput. Surv.*, 11(2):121–137, June 1979. ISSN 0360-0300. doi: 10.1145/356770.356776. URL <https://doi.org/10.1145/356770.356776>.
- Klaus Engel, Markus Hadwiger, Joe Michael Kniss, Aaron E. Lefohn, Christof Rezk Salama, and Daniel Weiskopf. Real-time volume graphics. In *International Conference on Computer Graphics and Interactive Techniques*, 2006. URL <https://api.semanticscholar.org/CorpusID:9495823>.
- Andrew S. Glassner. Space subdivision for fast ray tracing. *IEEE Computer Graphics and Applications*, 4(10):15–24, 1984. doi: 10.1109/MCG.1984.6429331.
- Aaron Jarmusch, Nathan Graddon, and Sunita Chandrasekaran. Dissecting the nvidia blackwell architecture with microbenchmarks, 2025. URL <https://arxiv.org/abs/2507.10789>.
- Arie Kaufman. Introduction to volume graphics. *Computers Graphics - CG*, 01 2000.
- Timothy L. Kay and James T. Kajiya. Ray tracing complex scenes. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '86, page 269–278, New York, NY, USA, 1986. Association for Computing Machinery. ISBN 0897911962. doi: 10.1145/15922.15916. URL <https://doi.org/10.1145/15922.15916>.
- John Kessenich, Dave Baldwin, and Randi Rost. *The OpenGL Shading Language, Version 4.60.8*. The Khronos Group Inc., 2019. URL <https://registry.khronos.org/OpenGL/specs/gl/GLSLangSpec.4.60.pdf>. Accessed: 2026-09-02.
- Ludwic Leonard. Rendervous: Physically-based differentiable renderer integrated with pytorch and vulkan. <https://github.com/rendervous/rendervous>, 2024a. Accessed: 2026-09-02.

- Ludwic Leonard. Vulky: Facade to vulkan api from python. https://github.com/rendervous/vulky_project, 2024b. Accessed: 2026-09-02.
- Ludwic Leonard, Nils Thuerey, and Rüdiger Westermann. Light transport-aware diffusion posterior sampling for single-view reconstruction of 3d volumes. In *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, pages 16163–16174, June 2025.
- N. Max. Optical models for direct volume rendering. *IEEE Transactions on Visualization and Computer Graphics*, 1(2):99–108, 1995. doi: 10.1109/2945.468400.
- Donald Meagher. Geometric modeling using octree-encoding. *Computer Graphics and Image Processing*, 19:129–147, 06 1982. doi: 10.1016/0146-664X(82)90104-6.
- Ken Museth. Vdb: High-resolution sparse volumes with dynamic topology. *ACM Trans. Graph.*, 32(3), July 2013. ISSN 0730-0301. doi: 10.1145/2487228.2487235. URL <https://doi.org/10.1145/2487228.2487235>.
- Ken Museth. Hierarchical digital differential analyzer for efficient ray-marching in openvdb. In *ACM SIGGRAPH 2014 Talks*, SIGGRAPH '14, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450329606. doi: 10.1145/2614106.2614136. URL <https://doi.org/10.1145/2614106.2614136>.
- Ken Museth. Nanovdb: A gpu-friendly and portable vdb data structure for real-time rendering and simulation. pages 1–2, 07 2021. doi: 10.1145/3450623.3464653.
- Ken Museth and Academy Software Foundation. NanoVDB: GPU-friendly sparse volumes. <https://github.com/AcademySoftwareFoundation/openvdb/tree/master/nanovdb>, 2024. Accessed: 2026-09-04.
- NVIDIA Corporation. *CUDA Programming Guide: Programming Model*. NVIDIA Corporation, 2026a. URL <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html>.
- NVIDIA Corporation. *NVIDIA Nsight Graphics Documentation*, 2026b. URL <https://docs.nvidia.com/nsight-graphics/>. Accessed: 2026-09-14.
- S. Parker, M. Parker, Y. Livnat, P.-P. Sloan, C. Hansen, and P. Shirley. Interactive ray tracing for volume visualization. *IEEE Transactions on Visualization and Computer Graphics*, 5(3):238–250, 1999. doi: 10.1109/2945.795215.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library, 2019. URL <https://arxiv.org/abs/1912.01703>.
- Wikipedia contributors. Trilinear interpolation. https://en.wikipedia.org/wiki/Trilinear_interpolation, 2024. Accessed: 2026-09-12.

Francis Williams, Jiahui Huang, Jonathan Swartz, Gergely Klar, Vijay Thakkar, Matthew Cong, Xuanchi Ren, Ruilong Li, Clement Fuji-Tsang, Sanja Fidler, Eftychios Sifakis, and Ken Museth. fvdb : A deep-learning framework for sparse, large scale, and high performance spatial intelligence. *ACM Transactions on Graphics*, 43(4):1–15, 2024. ISSN 1557-7368. doi: 10.1145/3658226. URL <http://dx.doi.org/10.1145/3658226>.